

**Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное
образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»**

Ереванский филиал

Кафедра Информационные технологии и гуманитарные науки

Дипломная Работа

**на тему «Разработка и реализация программного обеспечения для
встроенных систем»**

Выполнил
обучающийся группы ДКИ-2101
очной формы обучения
Амирханян А.Д.

Научный руководитель: Вирабян.Г.Б.

Ереван – 2026

Введение.....	3
Глава 1. Теоретические основы разработки программного обеспечения для встроженных систем.....	6
Глава 2. Инструменты и технологии разработки ПО для встроженных систем	22
Глава 3. Разработка и реализация программного обеспечения для встроженной системы.....	45
3.1 Постановка задачи и выбор аппаратной платформы.....	45
3.2 Проектирование архитектуры программного обеспечения.....	48
3.3 Реализация и тестирование программного обеспечения	56
Заключение	75
Список использованных источников	78

Введение

Современный этап развития информационных технологий характеризуется стремительным ростом числа устройств, функционирующих на базе встроенных систем. Встроенные системы находят широкое применение в промышленности, транспортной отрасли, телекоммуникациях, медицине, бытовой электронике и сфере Интернета вещей (IoT). Их использование позволяет автоматизировать технологические процессы, повысить надёжность оборудования, снизить энергопотребление и обеспечить высокую степень автономности функционирования технических устройств.

Актуальность темы исследования обусловлена возрастающей ролью встроенных систем в цифровой трансформации различных отраслей экономики, а также необходимостью разработки эффективного и надёжного программного обеспечения, учитывающего ограничения по ресурсам, требованиям к производительности и условиям реального времени. В отличие от традиционных программных решений для персональных компьютеров и серверов, программное обеспечение для встроенных систем разрабатывается с учётом строго ограниченных вычислительных ресурсов, требований к детерминированности выполнения и повышенной устойчивости к сбоям.

Разработка программного обеспечения для встроенных систем представляет собой сложный и многоэтапный процесс, включающий анализ требований, проектирование архитектуры, реализацию, тестирование и отладку. Особое значение приобретают вопросы оптимизации использования памяти, управления периферийными устройствами, обработки прерываний и обеспечения взаимодействия программных модулей. В ряде случаев требуется применение операционных систем реального времени, обеспечивающих корректное распределение задач и соблюдение временных ограничений.

Эффективная реализация программного обеспечения для встроенных систем позволяет обеспечить стабильную работу аппаратных компонентов, повысить надёжность функционирования устройств и расширить их функциональные возможности. При этом качество проектирования программной архитектуры напрямую влияет на масштабируемость и безопасность системы.

Целью данной работы является разработка и реализация программного обеспечения для встроенной системы с учётом требований к надёжности, производительности и рациональному использованию аппаратных ресурсов.

Для достижения поставленной цели предполагается решение следующих задач:

- исследовать теоретические основы построения встроенных систем и особенности их архитектуры
- изучить инструменты и технологии разработки программного обеспечения для встроенных платформ
- проанализировать особенности использования языков программирования и операционных систем реального времени во встроенных системах
- разработать архитектуру программного обеспечения для выбранной аппаратной платформы
- реализовать программное решение
- провести оценку эффективности и корректности функционирования разработанной системы

Объект исследования - встроенные системы как класс аппаратно-программных комплексов.

Предмет исследования - методы и средства разработки программного обеспечения для встроенных систем.

Методы исследования включают анализ научной и технической литературы, системный анализ архитектурных решений, методы проектирования программных систем, а также практическую реализацию программного обеспечения.

Структура работы включает введение, три главы, заключение, список использованных источников и приложения. В первой главе рассматриваются теоретические основы встроенных систем и особенности их архитектуры. Во второй главе анализируются инструменты и технологии разработки программного обеспечения для встроенных платформ. Третья глава посвящена практической разработке и реализации программного обеспечения для выбранной встроенной системы.

Глава 1. Теоретические основы разработки программного обеспечения для встроенных систем

1.1 Понятие и особенности встроенных систем

Встроенная система является специализированной вычислительной системой, предназначенной для управления определённым устройством или технологическим процессом и функционирующей как часть более сложного аппаратного комплекса. Иными словами, встроенная система представляет собой аппаратно-программный комплекс, выполняющий ограниченный набор задач в рамках конкретного устройства.

Согласно общепринятому определению, встроенная система - это «вычислительная система специального назначения, интегрированная в техническое устройство и предназначенная для выполнения заранее определённых функций, зачастую в режиме реального времени».

В отличие от универсальных вычислительных систем (персональных компьютеров или серверов), встроенные системы разрабатываются под конкретную задачу и, как правило, работают в условиях ограниченных вычислительных ресурсов, памяти и энергопотребления. Они могут функционировать автономно, без участия пользователя, и часто являются критически важными для корректной работы оборудования.

Примеры встроенных систем включают:

- микроконтроллеры в бытовой технике
- электронные блоки управления в автомобилях
- системы промышленной автоматизации
- медицинское оборудование
- устройства Интернета вещей (IoT)

Важно отметить, что встроенные системы охватывают широкий спектр областей - от простых микроконтроллеров до сложных распределённых систем на базе микропроцессоров с операционными системами.

Программное обеспечение встроенных систем

Программное обеспечение встроенной системы представляет собой совокупность программных модулей, обеспечивающих управление аппаратными компонентами, обработку входных сигналов и формирование управляющих воздействий.

Разработка ПО для встроенных систем существенно отличается от разработки программ для настольных или веб-приложений. Основные отличительные особенности включают:

- работу в условиях ограниченной оперативной и постоянной памяти
- необходимость строгого соблюдения временных ограничений (реальное время)
- непосредственное взаимодействие с аппаратными регистрами и периферийными устройствами
- повышенные требования к надёжности и отказоустойчивости
- минимизацию энергопотребления.

Программное обеспечение может функционировать:

- без операционной системы (bare-metal)
- под управлением операционной системы реального времени (RTOS)
- на базе встраиваемых версий Linux и других ОС.

1.2 Архитектура встроенных систем

Архитектура встроенной системы представляет собой совокупность аппаратных и программных компонентов, их структурную организацию и принципы взаимодействия, обеспечивающие выполнение заданных функций в условиях ограниченных ресурсов и требований реального времени. Понимание архитектурных принципов построения встроенных систем является необходимым условием для разработки эффективного и надёжного программного обеспечения.

Ниже рассматриваются классификация встроенных систем по аппаратной базе, их аппаратная архитектура, а также типовая организация программного стека.

Классификация встроенных систем на основе аппаратной базы

Встроенные системы реализуются на различных аппаратных платформах, выбор которых определяется требованиями к производительности, энергопотреблению, стоимости и степени интеграции. Принято выделять следующие основные классы аппаратных платформ.

- Микроконтроллеры (MCU) представляют собой интегральные схемы, объединяющие в едином корпусе процессорное ядро, оперативную и постоянную память, а также набор периферийных интерфейсов. Микроконтроллеры характеризуются низким энергопотреблением, и малой стоимостью, что делает их наиболее распространённым решением для задач управления в устройствах Интернета вещей, промышленной автоматике и бытовой электронике. Типичными представителями данного класса являются семейства ARM Cortex-M от STMicroelectronics, а также ESP32 от Espressif.
- Системы на кристалле (SoC, System-on-Chip) представляют собой интегральные схемы, объединяющие на одном полупроводниковом кристалле процессорное ядро или несколько ядер, контроллеры памяти, графические и сигнальные процессоры, коммуникационные

интерфейсы и иные специализированные блоки. SoC обеспечивают высокую степень интеграции при относительно небольших размерах и энергопотреблении. Данный класс платформ широко применяется в смартфонах, планшетах, телевизионных приставках и телекоммуникационном оборудовании. Примерами являются Qualcomm Snapdragon, Apple M-серии и MediaTek Dimensity.

- Программируемые логические интегральные схемы (FPGA, Field-Programmable Gate Array) принципиально отличаются от перечисленных выше платформ: вместо фиксированной логики они содержат массив конфигурируемых логических блоков, соединения между которыми задаются программно. Это позволяет реализовывать произвольные цифровые схемы, в том числе специализированные процессорные ядра, на уровне аппаратного описания. FPGA применяются в задачах, требующих экстремально низких задержек обработки сигналов, высокого параллелизма вычислений и реконфигурируемости - в телекоммуникациях, радарных системах, и финансовых вычислениях.

Аппаратная архитектура встроенных систем

Несмотря на разнообразие платформ, аппаратная архитектура встроенной системы в большинстве случаев включает следующие функциональные блоки: процессорное ядро, подсистему памяти, шинную инфраструктуру и набор периферийных устройств.

Процессорное ядро является центральным вычислительным элементом системы. В подавляющем большинстве современных встроенных систем применяется архитектура ARM, представленная семействами Cortex-M (для микроконтроллеров), Cortex-R (для систем реального времени с повышенными требованиями к надёжности) и Cortex-A (для высокопроизводительных встроенных платформ).

Характеристики ядра - тактовая частота, разрядность, наличие модуля аппаратной обработки вещественных чисел (FPU), кэш-памяти и поддержки виртуальной памяти (MMU) - определяют вычислительные возможности платформы.

Подсистема памяти во встроенных системах организована существенно иначе, чем в универсальных вычислительных системах. Принято различать следующие виды памяти: флеш-память (Flash) используется для хранения

программного кода и постоянных данных; статическое ОЗУ (SRAM) обеспечивает высокоскоростной доступ для хранения переменных, стека и сегмента heap; динамическое ОЗУ (DRAM, SDRAM, LPDDR) применяется в более производительных системах, требующих значительного объёма оперативной памяти; электрически перепрограммируемое постоянное запоминающее устройство (EEPROM) используется для хранения конфигурационных параметров и данных, изменяемых в процессе эксплуатации.

Объём памяти во встроенных системах, как правило, значительно ограничен: микроконтроллеры начального уровня располагают единицами килобайт ОЗУ и десятками килобайт флеш-памяти, тогда как высокопроизводительные SoC могут включать гигабайты LPDDR-памяти. Данное ограничение оказывает прямое влияние на подходы к разработке программного обеспечения: применяется статическое выделение памяти, минимизация использования динамического распределения, а также тщательный контроль фрагментации.

Шинная инфраструктура обеспечивает обмен данными между процессорным ядром, памятью и периферийными устройствами. В архитектуре ARM широко применяется шина AMBA (Advanced Microcontroller Bus Architecture), включающая высокоскоростную шину АНВ (Advanced High-performance Bus) для взаимодействия с памятью и DMA-контроллерами, а также периферийную шину APB (Advanced Peripheral Bus) с меньшей пропускной способностью для подключения медленных периферийных модулей. Контроллер прямого доступа к памяти (DMA) позволяет осуществлять пересылку данных между периферией и памятью без участия процессорного ядра, разгружая его для выполнения вычислительных задач.

Периферийные устройства и интерфейсы составляют важнейшую часть аппаратной архитектуры встроенной системы, обеспечивая её взаимодействие с внешним миром. К типовым периферийным блокам относятся: универсальные порты ввода-вывода (GPIO), таймеры и счётчики, аналого-цифровые (АЦП) и цифро-аналоговые (ЦАП) преобразователи, последовательные интерфейсы связи (UART, SPI, I²C, CAN, USB), сетевые контроллеры (Ethernet, Wi-Fi, Bluetooth) и интерфейсы отображения (MIPI, GMSL). Выбор состава периферии определяется целевым применением системы и непосредственно влияет на проектирование драйверного слоя программного обеспечения.

Программная архитектура встроенных систем

Программное обеспечение встроенных систем традиционно организуется в виде многоуровневой иерархической структуры, обеспечивающей разделение ответственности между компонентами и повышающей переносимость кода между аппаратными платформами.

Слой аппаратной абстракции (HAL, Hardware Abstraction Layer) является нижним уровнем программного стека и непосредственно взаимодействует с аппаратными регистрами, контроллерами прерываний и периферийными модулями. HAL предоставляет унифицированный программный интерфейс (API) для вышестоящих слоёв, скрывая платформозависимые детали реализации. Благодаря наличию HAL становится возможным перенос прикладного программного обеспечения на новую аппаратную платформу путём замены или модификации только нижнего слоя. В экосистеме STM32 роль HAL выполняют библиотеки STM32 HAL, в Zephyr RTOS аналогичная функция возложена на подсистему драйверов.

Слой драйверов реализует логику управления конкретными периферийными устройствами и обеспечивает их инициализацию, конфигурирование, передачу и приём данных, а также обработку прерываний. Драйвер взаимодействует с HAL или непосредственно с аппаратными регистрами и предоставляет более высокоуровневый интерфейс для протокольных стеков или прикладного кода. Корректная реализация драйверов с учётом атомарности операций и управления энергопотреблением является одним из ключевых аспектов разработки встроенного ПО.

Операционная система реального времени (RTOS) занимает промежуточное положение в программном стеке и предоставляет механизмы планирования задач, межзадачного взаимодействия, синхронизации и управления ресурсами. RTOS гарантирует детерминированное время реакции на внешние события, что критично для систем с жёсткими временными ограничениями. Подробно RTOS рассматриваются в разделе 2.2 данной работы. В ряде случаев - при минимальных требованиях к ресурсам или однозадачном сценарии - RTOS не применяется, а программное обеспечение строится на основе суперцикла (bare-metal, superloop) с обработкой событий через прерывания.

Прикладной уровень реализует целевую логику устройства: алгоритмы управления, обработку данных, протоколы взаимодействия с пользователем

и внешними системами. Качество проектирования прикладного уровня определяет функциональность, расширяемость и сопровождаемость всего программного решения.



Рисунок 1. Обобщённая многоуровневая архитектура программного обеспечения встроенной системы

Взаимосвязь аппаратной и программной архитектур

Архитектура программного обеспечения встроенной системы неразрывно связана с её аппаратной базой. Выбор микроконтроллера или SoC определяет доступный объём памяти, набор периферийных интерфейсов и ограничения по энергопотреблению, что в свою очередь влияет на возможность применения RTOS, выбор языка программирования и допустимую сложность прикладных алгоритмов. Так, для систем на базе Cortex-M0+ с 32 кБ Flash и 8 кБ RAM применение полноценной RTOS нецелесообразно, тогда как платформы на базе Cortex-M4/M7 с сотнями килобайт памяти допускают использование FreeRTOS, Zephyr и аналогичных решений.

Важным аспектом является также взаимодействие механизма прерываний аппаратной платформы с программным стеком. Вектор прерываний, приоритеты IRQ и контроллер NVIC (в архитектуре ARM Cortex-M) непосредственно влияют на проектирование обработчиков прерываний (interrupt handlers) и механизмов синхронизации в RTOS. Некорректная настройка приоритетов прерываний является распространённой причиной трудновоспроизводимых ошибок в многозадачных встроенных системах.

Таким образом, архитектура встроенной системы представляет собой тесно связанную совокупность аппаратных и программных компонентов.

Глубокое понимание как аппаратной платформы - её процессорного ядра, подсистемы памяти, шин и периферии, - так и принципов организации программного стека, является необходимым условием для проектирования надёжного и эффективного встроенного программного обеспечения. В следующем разделе рассматривается жизненный цикл разработки ПО для встроенных систем.

1.3 Жизненный цикл разработки программного обеспечения для встроенных систем

Жизненный цикл разработки программного обеспечения (ЖЦ ПО) представляет собой совокупность этапов, через которые проходит программный продукт от момента возникновения концепции до завершения разработки и поддержки. Во встроенных системах жизненный цикл обладает рядом существенных особенностей, обусловленных тесной взаимосвязью программного обеспечения с аппаратной платформой, ресурсными ограничениями, требованиями к надёжности и детерминированности функционирования, а также нередко - необходимостью соответствия отраслевым стандартам безопасности.

Этап анализа требований

Анализ требований является первым и одним из наиболее критичных этапов жизненного цикла. Ошибки, допущенные на данном этапе, обходятся на порядок дороже, чем ошибки, выявленные в ходе тестирования.

Требования к встроенной системе принято разделять на несколько категорий. Функциональные требования описывают, что система должна делать: набор поддерживаемых команд, алгоритмы обработки данных, протоколы взаимодействия с внешними устройствами. Нефункциональные требования определяют характеристики качества системы: допустимое время реакции на событие, максимальное энергопотребление, допустимый объём используемой памяти, температурный диапазон эксплуатации.

Ограничения задают рамки реализации: целевая аппаратная платформа, используемый язык программирования, применяемая RTOS, требования к сертификации.

Специфической категорией для встроенных систем являются требования реального времени - временные ограничения на выполнение определённых операций. Различают жёсткие (hard real-time) ограничения, нарушение которых приводит к отказу системы или опасным последствиям, и мягкие (soft real-time) ограничения, допускающие редкие нарушения без катастрофических последствий. Корректная классификация временных требований на данном этапе определяет выбор планировщика RTOS, приоритеты задач и механизмы синхронизации.

Для формализации требований применяются диаграммы вариантов использования (use case) а также диаграммы состояний (State Machine Diagram). При разработке под Zephyr RTOS типичным результатом данного этапа является детальная спецификация задач (threads), их приоритетов, стека, временных характеристик и используемых системных объектов синхронизации.

Этап проектирования архитектуры

На этапе архитектурного проектирования определяется высокоуровневая структура программной системы: декомпозиция на модули и компоненты, интерфейсы между ними, распределение функциональности по задачам RTOS, стратегии управления памятью и механизмы обработки ошибок.

Ключевым решением является выбор архитектурного паттерна. Для встроенных систем наиболее распространены следующие подходы. Многозадачная архитектура на базе RTOS предполагает декомпозицию функциональности на набор параллельно выполняемых задач, взаимодействующих через очереди сообщений (message queues), семафоры, мьютексы и разделяемую память. Данный подход обеспечивает модульность, масштабируемость и чёткое разделение ответственности, однако требует тщательного проектирования для исключения дедлоков, состояний гонки (race condition) и инверсии приоритетов. Архитектура на основе конечных автоматов (FSM) широко применяется для управляющих систем с чётко определёнными состояниями и переходами. Иерархические конечные автоматы (HFSM) позволяют моделировать сложное поведение без экспоненциального роста числа состояний. Событийно-управляемая архитектура строится вокруг очереди событий и диспетчера, что обеспечивает низкую связанность компонентов и упрощает тестирование.

Важным аспектом архитектурного проектирования во встроенных системах является управление памятью. В условиях ограниченного ОЗУ применяется статическое выделение памяти для критически важных структур данных, использование пулов памяти фиксированного размера вместо универсального динамического аллокатора, а также тщательный расчёт размера стека каждой задачи во избежание переполнения. В экосистеме Zephyr RTOS данные аспекты решаются средствами подсистем `k_mem_pool`, `k_heap` и статическими конфигурационными макросами `K_THREAD_DEFINE`.

Результатом данного этапа является архитектурная документация, включающая диаграммы компонентов, диаграммы взаимодействия задач, описание интерфейсов модулей и обоснование принятых проектных решений.

Этап детального проектирования и реализации

Детальное проектирование конкретизирует архитектурные решения до уровня отдельных программных модулей, функций и структур данных. На данном этапе разрабатываются детальные спецификации модулей, определяются алгоритмы и структуры данных, проектируются обработчики прерываний и интерфейсы драйверов.

Реализация во встроенных системах ведётся преимущественно на языках C и C++, реже - на языках ассемблера для критически чувствительных ко времени фрагментов. Применение современных стандартов - C11, C++17 - позволяет использовать средства статического анализа и улучшает переносимость кода.

Особого внимания требует реализация обработчиков прерываний (ISR). ISR должны быть максимально краткими: в них допустимо лишь установить флаг, разместить данные в очередь или активировать семафор, тогда как основная обработка выносится в контекст задачи. Это правило продиктовано необходимостью минимизации времени, в течение которого прерывания заблокированы, и снижения задержки реакции системы на другие события. В Zephyr RTOS взаимодействие между ISR и задачами осуществляется через объекты `k_msgq`, `k_sem` и `k_fifo`, поддерживающие вызов из контекста прерывания.

Управление конфигурацией и системой сборки (build system) является неотъемлемой частью этапа реализации. Современные встроенные проекты используют системы сборки CMake в сочетании с системой конфигурации Kconfig (как в Zephyr), что позволяет гибко управлять составом компонентов и параметрами компиляции. Системы контроля версий (Subversion, Mercurial, Git) обеспечивают управляемость разработки, особенно в командных проектах. Непрерывная интеграция (CI) с автоматической сборкой и запуском тестов на каждый коммит позволяет оперативно выявлять регрессии.

Этап верификации и тестирования

Тестирование встроенного программного обеспечения существенно сложнее, чем тестирование прикладного ПО для универсальных платформ, ввиду зависимости от аппаратного обеспечения, наличия временных ограничений и трудности воспроизведения ряда особых условий (edge cases).

Модульное тестирование (unit testing) предполагает изолированную проверку отдельных функций и модулей. Для встроенных систем применяются фреймворки, поддерживающие кросс-компиляцию и выполнение тестов как на целевом оборудовании, так и на хост-машине (нативная сборка): CppUTest, Unity, Zephyr Twister. Использование техники заглушек (mocks, stubs) позволяет изолировать тестируемый модуль от аппаратнозависимых зависимостей.

Интеграционное тестирование проверяет корректность взаимодействия модулей между собой и с периферийными устройствами. На данном этапе неизбежно использование реального.

Тестирование в реальном времени направлено на проверку соответствия системы временным требованиям. Для измерения времени реакции, задержек планировщика и джиттера применяются аппаратные осциллографы, логические анализаторы и инструменты трассировки - SystemView от SEGGER, Percepio Tracealyzer. Превышение дедлайна задачи в системе с жёсткими требованиями реального времени является критической ошибкой и должно быть выявлено до развёртывания.

Статический анализ кода позволяет выявить потенциальные ошибки без выполнения программы. Инструменты - PC-lint, Polyspace, Coverity, clang-tidy - проверяют соответствие стандартам кодирования, обнаруживают утечки ресурсов, разыменованное нулевых указателей (null pointer dereferencing), переполнения буферов (buffer overflow) и иные распространённые уязвимости. Статический анализ особенно важен при разработке ПО для сертифицируемых систем: ряд стандартов безопасности требует его обязательного применения.

Этап отладки

Отладка встроенного ПО является специфическим и трудоёмким процессом, требующим специализированного инструментария. В отличие от отладки прикладного ПО, разработчик встроенных систем лишён ряда привычных инструментов и вынужден учитывать влияние самого процесса отладки на поведение системы реального времени.

Основным инструментом отладки является аппаратный отладчик - JTAG или SWD (Serial Wire Debug) интерфейс, обеспечивающий доступ к ядру процессора, регистрам, памяти и периферии без вмешательства в выполняемый код. Стандартные решения - J-Link от SEGGER, ST-LINK, CMSIS-DAP - интегрируются с IDE (VS Code с расширением Cortex-Debug, CLion, Ozone) и обеспечивают пошаговое выполнение, точки останова, наблюдение за переменными и профилирование.

Полупостоянный вывод (printf-debugging) через UART или RTT (Real-Time Transfer от SEGGER) остаётся практически значимым инструментом диагностики, позволяя регистрировать состояние системы без остановки выполнения. RTT обеспечивает минимальное влияние на временные характеристики системы и может применяться даже в системах с жёсткими требованиями реального времени. Zephyr предоставляет подсистему логирования (LOG_INF, LOG_ERR и т.д.) с поддержкой нескольких бэкендов, в том числе RTT и UART и с возможностью имплементации собственного бэкенда (подробнее описано в главе 3.3).

Аппаратные инструменты - осциллографы, логические анализаторы, анализаторы протоколов - незаменимы при отладке взаимодействия с периферией: синхронизацией SPI, временными диаграммами I²C, целостностью сигналов на высоких частотах. При отладке беспроводных протоколов, таких как LTE-M и NB-IoT в контексте nRF9161, применяются специализированные анализаторы сотовых протоколов и инструменты трассировки модема - nRF Connect for Desktop с модулем Trace Collector.

Этап программирования и обновления прошивки

Способы программирования варьируются от прямой записи через JTAG/SWD на этапе производства до беспроводного обновления прошивки (OTA, Over-the-Air) в процессе эксплуатации.

Механизм обновления прошивки по воздуху (FOTA, Firmware Over-the-Air) приобретает критическое значение для IoT-устройств, работающих в труднодоступных местах в большом количестве. FOTA позволяет

исправлять ошибки, добавлять функциональность и обновлять сертификаты безопасности без физического доступа к устройству. Zephyr RTOS предоставляет подсистему MCUboot - открытый загрузчик с поддержкой безопасной загрузки (Secure Boot), проверки целостности и атомарного обновления прошивки. Для nRF9161 механизм FOTA реализуется через LTE-M/NB-IoT с использованием сервиса nRF Connect Device Management и протокола HTTPS.

Обязательным элементом надёжного FOTA является механизм отката (rollback): в случае если новая прошивка не прошла верификацию после установки или устройство не вышло на связь в течение заданного тайм-аута, загрузчик автоматически восстанавливает предыдущую работоспособную версию. MCUboot реализует данный механизм через слоты образов прошивки (primary/secondary slot) и статус подтверждения (image confirmed).

Этап поддержки и управления версиями

Поддержка встроенного ПО включает исправление выявленных дефектов, адаптацию к изменениям аппаратной платформы, обновление сторонних зависимостей (HAL, RTOS, SDK) и добавление новой функциональности. Особую сложность представляет долгосрочная поддержка систем с большим количеством устройств в эксплуатации: необходимо поддерживать совместимость новых версий прошивки с различными ревизиями аппаратного обеспечения.

Управление версиями в профессиональных встроенных проектах охватывает не только исходный код, но и конфигурационные файлы сборки, схемотехническую документацию, тестовые сценарии и отчёты о верификации. Семантическое версионирование (Semantic Versioning) обеспечивает однозначную интерпретацию номеров версий, а инструменты автоматической генерации журнала изменений (CHANGELOG) упрощают сопровождение документации.

Управление зависимостями от внешних компонентов - версий RTOS, SDK производителя, сторонних библиотек - является нетривиальной задачей. Обновление версии Zephyr или nRF Connect SDK может потребовать адаптации кода в связи с изменениями API. Практика фиксации точных версий зависимостей (pinning) в сочетании с воспроизводимыми сборочными окружениями (Docker контейнеры) позволяет обеспечить стабильность сборочного процесса.

Особенности жизненного цикла в контексте функциональной безопасности

Таким образом, жизненный цикл разработки программного обеспечения для встроенных систем представляет собой комплексный и многоэтапный процесс, существенно отличающийся от разработки прикладного ПО. Тесная взаимосвязь с аппаратной платформой, требования реального времени, и ограниченность ресурсов обуславливают применение специализированных методологий, инструментов и практик на каждом этапе. Глубокое понимание жизненного цикла является необходимой основой для принятия обоснованных архитектурных и технологических решений, рассматриваемых в последующих главах настоящей работы.

Глава 2. Инструменты и технологии разработки ПО для встроенных систем

2.1 Языки программирования во встроенных системах

Выбор языка программирования является одним из фундаментальных решений при разработке программного обеспечения для встроенных систем. В отличие от прикладного программирования, где выбор языка во многом определяется экосистемой и предпочтениями команды, в области встроенных систем данное решение непосредственно влияет на эффективность использования аппаратных ресурсов, детерминированность поведения программы, возможности отладки и переносимость кода между платформами. Исторически сложился относительно узкий круг языков, доминирующих в данной области, прежде всего С и ассемблер, однако в последние годы наблюдается постепенное проникновение С++ в отдельные сегменты рынка встроенных систем.

Далее будут анализироваться основные языки программирования, применяемые во встроенных системах, рассматриваться их достоинства и ограничения.

Язык ассемблера

Исторически первым языком программирования встроенных систем являлся ассемблер - язык символического кодирования машинных инструкций конкретного процессора. Программирование на ассемблере обеспечивает полный контроль над каждой машинной командой, регистрами процессора, флагами состояния и тактовыми циклами выполнения. Это делает ассемблер незаменимым инструментом в случаях, когда требуется абсолютная предсказуемость времени выполнения отдельных операций или максимальная оптимизация критически важных фрагментов кода.

В современной разработке встроенного ПО ассемблер применяется в строго ограниченном круге задач. Прежде всего, это низкоуровневый код начальной инициализации системы - процедура сброса (reset handler), инициализация стека, настройка таблицы векторов прерываний и переход к функции main. В архитектуре ARM Cortex-M данный код реализован в файлах startup_*.s, предоставляемых производителем микроконтроллера. Кроме того, ассемблер применяется для реализации переключения контекста в ядре RTOS - операции сохранения и восстановления регистрового состояния задач, которая по своей природе является аппаратнозависимой и не может быть реализована средствами языка C в полной мере. Zephyr RTOS содержит ассемблерные файлы для реализации переключения контекста на каждой поддерживаемой архитектуре, в том числе для ARM Cortex-M33, на котором основан nRF9161.

Вместе с тем применение ассемблера в качестве основного языка разработки нецелесообразно по ряду причин. Код на ассемблере жёстко привязан к конкретной архитектуре процессора и не переносим между платформами. Сопровождение и модификация ассемблерного кода требуют значительно больших трудозатрат, чем аналогичные операции с кодом на языках высокого уровня. Производительность разработки существенно ниже, а вероятность ошибок - выше. По этим причинам ассемблер занимает вспомогательную роль в современных проектах встроенного ПО, ограничиваясь несколькими десятками или сотнями строк в проектах, насчитывающих тысячи строк кода на C/C++.

Язык C как основной инструмент разработки встроенных систем

Язык программирования C занимает доминирующее положение в области встроенных систем на протяжении нескольких десятилетий и по сей день остаётся наиболее широко применяемым языком в данной сфере. Согласно результатам отраслевых опросов - в частности, исследования Embedded.com State of the Nation Survey - язык C стабильно занимает первое место по популярности среди разработчиков встроенного ПО, значительно опережая ближайших конкурентов.

Доминирование C во встроенных системах обусловлено совокупностью факторов, делающих его исключительно хорошо приспособленным для данной области.

Близость к аппаратуре. Язык C предоставляет прямой доступ к аппаратным ресурсам через механизм указателей и операции с памятью. Доступ к регистрам периферийных устройств реализуется через разыменование указателя на адрес регистра с квалификатором `volatile`, предотвращающим нежелательные оптимизации компилятора:

```
#include <stdint.h>

#define UART0_DR (*(volatile uint32_t * const)0x40004000U))

int main()
{
    UART0_DR = 'A';
    return 0;
}
```

Данный механизм обеспечивает предсказуемый и прямой доступ к регистрам без накладных расходов, характерных для языков высокого уровня. В экосистеме Zephyr RTOS прямой доступ к регистрам абстрагирован через макросы `sys_read32` / `sys_write32` и заголовочные файлы

Device Tree, однако в основе по-прежнему лежат операции с указателями на языке C.

```
// Fragment that configures an imx219

cam_node: imx219@10 {
    compatible = "sony,imx219";
    reg = <0x10>;
    status = "disabled";

    clocks = <&cam1_clk>;
    clock-names = "xclk";

    VANA-supply = <&cam1_reg>; /* 2.8v */
    VDIG-supply = <&cam_dummy_reg>; /* 1.8v */
    VDDL-supply = <&cam_dummy_reg>; /* 1.2v */

    rotation = <180>;
    orientation = <2>;

    port {
        cam_endpoint: endpoint {
            clock-lanes = <0>;
            data-lanes = <1 2>;
            clock-noncontinuous;
            link-frequencies =
                /bits/ 64 <456000000>;
        };
    };
};
```

Предсказуемость и детерминированность. Семантика языка C, при условии соблюдения стандартов кодирования и избегания неопределённого поведения, обеспечивает достаточно высокую предсказуемость генерируемого машинного кода.

Эффективность компилируемого кода. Современные компиляторы C - GCC, Clang/LLVM - генерируют высокооптимизированный машинный код, по производительности сопоставимый с ручным ассемблером в большинстве сценариев. Флаги оптимизации (-O2, -Os для минимизации размера кода) позволяют гибко управлять соотношением производительности и размера исполняемого файла.

Минимальные накладные расходы времени выполнения. В отличие от языков с управляемой памятью (Java, Python, C#), язык C не предполагает обязательного наличия среды выполнения, сборщика мусора или виртуальной машины. Программа на C может быть запущена непосредственно после минимальной процедуры инициализации, что делает его применимым даже для микроконтроллеров с единицами килобайт оперативной памяти.

Большая экосистема и широкая поддержка платформ. Практически каждый производитель микроконтроллеров предоставляет SDK, HAL и примеры кода на языке C. Zephyr RTOS написан преимущественно на C и предоставляет API, ориентированный на C. nRF Connect SDK от Nordic Semiconductor, включающий поддержку nRF9161, также основан на C с использованием расширений GNU. Это означает, что вся документация, примеры, драйверы и системные библиотеки написаны на C, что существенно снижает порог вхождения и ускоряет разработку.

Применение языка C во встроенных системах имеет ряд специфических аспектов, существенно отличающих его от использования C в системном или прикладном программировании.

Квалификатор `volatile` является одним из наиболее важных и часто неправильно понимаемых инструментов в арсенале разработчика встроенного ПО. Он сообщает компилятору, что значение переменной может изменяться непредвиденным образом - из обработчика прерывания, из другого потока исполнения или вследствие аппаратного события, - и запрещает оптимизации, предполагающие неизменность значения переменной между последовательными обращениями. Его корректное применение критично при работе с регистрами периферии и переменными, разделяемыми между ISR и основным кодом.

Управление памятью в C для встроенных систем требует особой дисциплины. Динамическое выделение памяти через malloc/free в ряде случаев нежелательно: оно вносит недетерминированность во время выполнения затрудняет анализ потребления памяти.

Целочисленные типы фиксированной ширины, определённые в заголовочном файле <stdint.h> стандарта C99 - uint8_t, int16_t, uint32_t и т.д. - являются стандартной практикой во встроенном программировании. Использование платформозависимых типов int и long недопустимо при написании переносимого кода, поскольку их размер определяется реализацией компилятора и может различаться на разных архитектурах. Аналогично, <stdbool.h> предоставляет тип bool для явного представления булевых значений.

Битовые поля и операции с битами широко применяются для работы с регистрами периферийных устройств и компактного хранения флагов состояния. Маскирование, установка и сброс отдельных битов через операторы &, |, ^ и ~ являются ежедневной практикой разработчика встроенного ПО. Для структурированного доступа к битовым полям регистров применяются как структуры с битовыми полями (с учётом платформозависимости их укладки).

Атрибуты компилятора - расширения GNU GCC, широко применяемые в системном программировании на C, - позволяют управлять размещением функций и данных в памяти, выравниванием, и иными характеристиками генерируемого кода.

- Атрибут __attribute__((packed)) исключает выравнивание структуры, что важно при работе с битовыми форматами данных.
- Атрибут __attribute__((constructor)) заставляет компилятор выполнять функцию автоматически до вызова основной функции

main(). Это критически важно для инициализации модулей, драйверов или глобальных объектов встраиваемых систем еще на этапе старта программы

- Атрибут `__attribute__((cold))` Указывает компилятору, что функция вызывается крайне редко (например, обработка критических ошибок или сброс системы). Компилятор оптимизирует этот код по размеру, а не по скорости, и выносит его в отдельную область памяти, чтобы он не забивал кэш инструкций процессора при выполнении основного кода.

Экосистема библиотек для языка C во встроенных системах

Одним из существенных преимуществ языка C в контексте встроенных систем является богатая экосистема библиотек, адаптированных к условиям ограниченных ресурсов. В отличие от экосистем языков высокого уровня, где библиотеки нередко рассчитаны на неограниченную память и наличие операционной системы общего назначения, библиотеки для встроенного C проектируются с явным учётом отсутствия динамической аллокации, минимального объёма ОЗУ и детерминированности поведения. В данном контексте особого внимания заслуживает паттерн `single-header` библиотек - подход к организации и распространению программных компонентов, при котором вся реализация библиотеки сосредоточена в единственном заголовочном файле с расширением `.h`. Данный подход был популяризирован Шоном Барреттом (Sean Barrett) в рамках проекта `stb` - коллекции утилитарных библиотек для языка C, получившей широкое распространение в сообществе разработчиков игр и системного ПО. Философия `stb`-библиотек формулируется следующим образом: библиотека должна быть настолько простой в интеграции, чтобы для её использования было достаточно скопировать один файл в директорию проекта.

Механизм работы single-header библиотеки основан на условной компиляции: заголовочный файл содержит как объявления (declarations), так и определения (definitions), причём последние включаются в единицу трансляции только в зависимости от специального макроса-активатора:

```
// В одном .с файле проекта – активация реализации:  
#include "jsmn.h"  
  
// В остальных файлах – только объявления:  
#define JSMN_HEADER  
#include "jsmn.h"
```

Рисунок 2

Язык C++ во встроенных системах

Язык C++ находит применение во встроенных системах с достаточным объёмом ресурсов, прежде всего на платформах класса Cortex-M3 и выше. Его главным преимуществом по сравнению с C является поддержка объектно-ориентированного программирования, обобщённого программирования (шаблоны), и RAII (Resource Acquisition Is Initialization) - идиомы автоматического управления ресурсами через деструкторы.

Вместе с тем применение C++ во встроенных системах требует осознанного подхода и ограничения использования ряда возможностей языка. Динамическое связывание (dynamic dispatch) через виртуальные функции вносит накладные расходы на разыменование таблицы виртуальных методов (vtable) и может приводить к недетерминированному поведению кэша инструкций. Исключения C++ (exceptions) категорически нежелательны во встроенных системах: их обработка требует значительного объёма кода и таблиц раскрутки стека (unwind tables), которые увеличивают размер исполняемого файла и вносят непредсказуемые задержки. Стандартная библиотека C++ (STL) в полном объёме неприменима на платформах с ограниченной памятью ввиду интенсивного использования динамического выделения памяти. Для встроенных систем разработаны альтернативные библиотеки - ETL (Embedded Template Library), предоставляющая аналоги контейнеров STL на статической памяти.

Подмножество C++, рекомендуемое для встроенных систем, включает классы без виртуальных методов, шаблоны, пространства имён, constexpr-вычисления времени компиляции и ссылки (references) как более безопасную альтернативу указателям (pointers). Компиляция с

флагами `-fno-exceptions` `-fno-rtti` отключает исключения и динамическую информацию о типах, сокращая размер бинарного файла.

Сравнительный анализ и обоснование выбора языка C++

Для наглядного сопоставления рассмотренных языков приведём их ключевые характеристики применительно к требованиям встроенных систем.

Язык ассемблера обеспечивает максимальный контроль над аппаратурой и нулевые накладные расходы, однако практически не переносим, крайне трудоёмок в разработке и сопровождении, и применяется исключительно для строго ограниченных низкоуровневых задач.

Язык C сочетает близость к аппаратуре, предсказуемость и эффективность генерируемого кода с достаточным уровнем абстракции для разработки сложных программных систем. Широкая экосистема, богатая документация, широкая поддержка производителями платформ и наличие развитых стандартов кодирования делают C оптимальным выбором для встроенных проектов.

Язык C++ расширяет возможности C средствами объектно-ориентированного и обобщённого программирования.

Принимая во внимание, что практическая часть работы реализуется на платформе nRF9161 с использованием Zephyr RTOS и nRF Connect SDK, выбор языка C++ открывает дополнительные возможности для архитектурного проектирования. Современный C++ полностью поддерживается инструментарием Zephyr, а его использование позволяет внедрить объектно-ориентированный подход, инкапсуляцию и абстракции поверх существующей экосистемы ядра. Инфраструктура Zephyr (система сборки, Kconfig, Device Tree и низкоуровневые API драйверов)

бесшовно интегрируется с C++, обеспечивая компиляцию без потери производительности. Такой выбор позволяет совместить богатую базу примеров экосистемы nRF с передовыми практиками проектирования сложного и масштабируемого встроенного ПО.

Таким образом, выбор языка программирования для встроенных систем является многокритериальным решением, учитывающим аппаратную платформу, требования к ресурсам, временные ограничения, требования к сертификации и зрелость экосистемы. Язык C++, обладая уникальным сочетанием близости к аппаратуре, предсказуемости поведения и зрелости инструментальной поддержки, является оптимальным языком разработки встроенного ПО и служит основным инструментом в практической части настоящей работы. В следующем разделе рассматриваются операционные системы реального времени как ключевой программный компонент встроенных систем.

2.2 Операционные системы реального времени (RTOS)

Операционная система реального времени представляет собой специализированную операционную систему, предназначенную для управления аппаратными ресурсами и планирования выполнения задач в условиях жёстких временных ограничений. Ключевым отличием RTOS от операционных систем общего назначения является не столько высокая скорость выполнения операций, сколько их детерминированность - гарантия того, что реакция системы на внешнее событие произойдёт в пределах заранее известного и гарантированного временного интервала. Именно это свойство делает RTOS незаменимым компонентом встроенных систем управления, телекоммуникационного оборудования, медицинских приборов и устройств Интернета вещей.

Фундаментальные концепции операционных систем реального времени

В теории встроенных систем принято различать три категории систем реального времени в зависимости от последствий нарушения временных ограничений.

Системы с жёсткими требованиями реального времени (hard real-time) допускают нарушение дедлайна как катастрофическое событие, приводящее к отказу системы или опасным последствиям. Классическими примерами являются системы управления двигателем автомобиля, авиационные бортовые компьютеры, медицинские имплантируемые устройства и системы управления промышленными роботами. Для таких систем необходимо формальное доказательство соблюдения всех временных ограничений при наихудшем сценарии выполнения (Worst-Case Execution Time, WCET).

Системы с мягкими требованиями реального времени (soft real-time) допускают редкие нарушения дедлайна без катастрофических последствий, хотя и с деградацией качества функционирования. Примерами являются системы потоковой передачи аудио и видео, пользовательские интерфейсы и телекоммуникационные протоколы без гарантий качества обслуживания.

Системы с твёрдыми требованиями реального времени (firm real-time) занимают промежуточное положение: результат, полученный после истечения дедлайна, является бесполезным, однако его отсутствие не приводит к катастрофическим последствиям. Примером является система обработки биржевых котировок, где устаревшие данные не несут ценности, но их отсутствие не вызывает аварии.

Планировщик задач и алгоритмы планирования

Планировщик (scheduler) является центральным компонентом ядра RTOS, определяющим, какая задача получает процессорное время в каждый момент. Алгоритм планирования непосредственно влияет на детерминированность системы и соблюдение временных ограничений.

Планирование с фиксированными приоритетами является наиболее распространённым подходом в RTOS. Каждой задаче назначается числовой приоритет, и планировщик всегда передаёт управление готовой к выполнению задаче с наивысшим приоритетом. При появлении готовой задачи с приоритетом выше текущей выполняющейся происходит вытеснение (preemption) - немедленное прерывание текущей задачи и

передача управления более приоритетной. Данная модель реализована в подавляющем большинстве RTOS, включая FreeRTOS и Zephyr.

Примитивы синхронизации и межзадачного взаимодействия

Параллельное выполнение нескольких задач неизбежно порождает необходимость координации доступа к разделяемым ресурсам и организации обмена данными между задачами. RTOS предоставляет набор примитивов синхронизации, каждый из которых имеет определённую область применения.

Семафор (semaphore) представляет собой счётчик с атомарными операциями инкремента и декремента. Бинарный семафор используется для сигнализации между задачами или из обработчика прерывания в задачу - например, ISR устанавливает семафор по завершении приёма данных по DMA, а задача обработки данных ожидает его. Счётный семафор применяется для ограничения доступа к пулу ресурсов фиксированного размера.

Мьютекс (mutex, mutual exclusion) обеспечивает взаимоисключающий доступ к разделяемому ресурсу. В отличие от бинарного семафора, мьютекс поддерживает концепцию владения: только захватившая мьютекс задача вправе его освободить. Корректно реализованный мьютекс в RTOS поддерживает наследование приоритетов (priority inheritance) - механизм предотвращения инверсии приоритетов, при котором низкоприоритетная задача, удерживающая мьютекс, временно повышает свой приоритет до уровня наиболее приоритетной ожидающей задачи.

Инверсия приоритетов является классической проблемой систем с фиксированными приоритетами и мьютексами. Она возникает когда высокоприоритетная задача H ожидает мьютекс, удерживаемый низкоприоритетной задачей L, тогда как задача со средним приоритетом M вытесняет L и не позволяет ей освободить мьютекс. В результате H эффективно блокируется задачей M, имеющей более низкий приоритет. Данная проблема получила широкую известность благодаря инциденту с марсоходом Mars Pathfinder в 1997 году, где инверсия приоритетов привела к многократным перезагрузкам бортового компьютера. Наследование приоритетов, реализованное в Zephyr и других современных RTOS, устраняет данную проблему.

Очередь сообщений (message queue) обеспечивает асинхронный обмен данными между задачами по принципу FIFO (first-in first-out). Отправитель помещает сообщение в очередь и продолжает выполнение, не ожидая обработки; получатель извлекает сообщения в порядке поступления. Очереди сообщений являются предпочтительным механизмом передачи данных из ISR в задачу, поскольку операция помещения в очередь является атомарной и допустимой в контексте прерывания.

Таймеры программного обеспечения (software timers) позволяют запланировать вызов функции обратного вызова через заданный интервал времени - однократно или периодически. В отличие от аппаратных таймеров, программные таймеры RTOS мультиплексуют несколько виртуальных таймеров на один аппаратный, что позволяет создавать произвольное количество таймеров без дефицита аппаратных ресурсов.

Распространённые RTOS

FreeRTOS является наиболее широко используемой RTOS в мире встроенных систем. Разработанная Ричардом Барри и в 2017 году переданная под управление Amazon Web Services, FreeRTOS отличается минималистичным ядром, портируемым на сотни аппаратных платформ, простым и хорошо документированным API, а также открытым исходным кодом под лицензией MIT. Минимальный объём памяти, требуемый для работы ядра FreeRTOS, составляет порядка 6–12 кБ Flash и 1–2 кБ RAM, что делает её применимой даже на микроконтроллерах начального уровня.

RT-Thread - RTOS китайского происхождения с открытым исходным кодом, получившая широкое распространение в азиатском регионе и набирающая популярность глобально. Отличается богатой экосистемой компонентов - файловые системы, сетевые стеки, GUI-фреймворки - и поддержкой концепции «папо» версии для микроконтроллеров с минимальными ресурсами.

ThreadX (ныне Eclipse ThreadX, ранее Azure RTOS) - коммерческая RTOS от Express Logic, приобретённая Microsoft и впоследствии переданная в открытый доступ под управление Eclipse Foundation.

Mbed OS - RTOS и платформа разработки от ARM, ориентированная на IoT-устройства на базе Cortex-M. Предоставляет высокоуровневые абстракции для работы с сетью, файловой системой и периферией, однако требует значительно большего объёма ресурсов по сравнению с FreeRTOS.

Zephyr RTOS: архитектура, возможности и применение

Zephyr RTOS занимает особое место среди современных операционных систем реального времени. Проект был основан компанией Wind River Systems на базе ядра Rocket RTOS и в 2016 году передан Linux Foundation в качестве проекта с открытым исходным кодом. С тех пор Zephyr развивается при активном участии крупнейших игроков индустрии - Intel, Nordic Semiconductor, NXP, Google, Meta и многих других - и сегодня является одним из наиболее динамично развивающихся проектов в области встроенных систем.

Архитектура ядра Zephyr

Ядро Zephyr реализует многопоточный планировщик с приоритетами. Архитектура ядра предусматривает чёткое разделение между потоками (с отрицательными приоритетами в терминологии Zephyr), выполняющимися без вытеснения до явной передачи управления, и вытесняемыми потоками (с неотрицательными приоритетами), допускающими принудительное прерывание планировщиком.

Zephyr поддерживает как статическое, так и динамическое создание потоков. Статическое создание посредством макроса `K_THREAD_DEFINE` является предпочтительным: поток и его стек создаются на этапе компиляции, что обеспечивает полную предсказуемость потребления памяти и исключает возможность отказа создания потока в процессе выполнения:

```
#define STACK_SIZE 1024
#define THREAD_PRIORITY 5

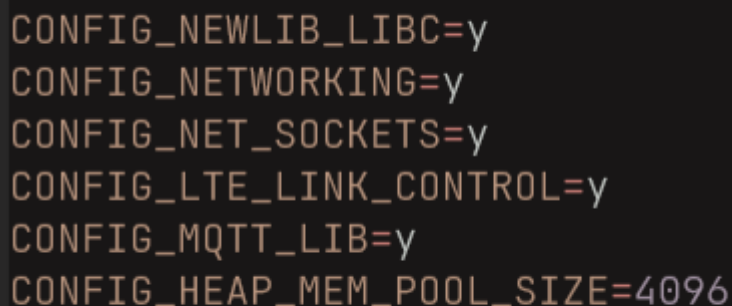
K_THREAD_DEFINE(sensor_thread_id,
                STACK_SIZE,
                sensor_thread_entry,
                NULL, NULL, NULL,
                THREAD_PRIORITY,
                0,
                0);
```

Рисунок 3

Система конфигурации Kconfig и Device Tree

Одной из наиболее отличительных особенностей Zephyr является использование двух взаимодополняющих механизмов конфигурации, заимствованных из ядра Linux.

Система Kconfig обеспечивает конфигурацию программных компонентов ядра: включение и отключение подсистем, задание параметров планировщика, размеров буферов и иных числовых констант. Конфигурация задаётся в файле `prj.conf` проекта в виде пар ключ-значение (key-value) и преобразуется в заголовочный файл `autosconf.h`, включаемый в процессе компиляции:



```
CONFIG_NEWLIB_LIBC=y
CONFIG_NETWORKING=y
CONFIG_NET_SOCKETS=y
CONFIG_LTE_LINK_CONTROL=y
CONFIG_MQTT_LIB=y
CONFIG_HEAP_MEM_POOL_SIZE=4096
```

Рисунок 4

Механизм Device Tree (DTS) описывает аппаратную конфигурацию системы - набор периферийных устройств, их адреса, прерывания и параметры инициализации - в декларативном текстовом формате, независимом от исходного кода. Это позволяет одному и тому же программному коду работать на различных аппаратных платформах без изменений: достаточно предоставить соответствующий файл описания платы (board overlay). Для nRF9161-DK в составе nRF Connect SDK предоставляется полный набор DTS-файлов, описывающих все периферийные устройства, включая модем LTE, интерфейс UART, GPIO и встроенный АЦП. Доступ к устройствам в коде осуществляется через макросы `DEVICE_DT_GET` и `DT_NODELABEL`, обеспечивающие проверяемое на этапе компиляции получение дескриптора устройства:

```
static const struct device *uart_dev =  
    DEVICE_DT_GET(DT_NODELABEL(uart0));
```

Рисунок 5

Подсистема драйверов и модель устройств

Zephyr реализует унифицированную модель устройств, в которой каждый драйвер периферии представлен структурой `device`, содержащей указатель на таблицу операций и указатель на конфигурационные данные экземпляра. Взаимодействие с устройствами осуществляется через стандартизированные API подсистем - `gpio.h`, `uart.h`, `spi.h`, `i2c.h`, - что обеспечивает переносимость прикладного кода между платформами при сохранении платформозависимой реализации на уровне драйверов.

Система сборки и инструментарий

Zephyr использует CMake в качестве системы сборки верхнего уровня и инструмент `west` - мета-инструмент управления рабочим пространством, объединяющий функции управления зависимостями, сборки, прошивки и отладки. nRF Connect SDK от Nordic Semiconductor представляет собой расширение Zephyr, добавляющее поддержку семейства nRF91, nRF53 и nRF52, библиотеки для работы с модемом и протоколом LTE.

Преимущества Zephyr для разработки на nRF9161

Выбор Zephyr RTOS в качестве операционной системы для проекта на базе nRF9161 обусловлен совокупностью факторов. Во-первых, nRF Connect SDK - официальный SDK Nordic Semiconductor для nRF9161 - основан на Zephyr и предоставляет все необходимые драйверы, библиотеки модема и примеры приложений исключительно в экосистеме Zephyr. Во-вторых, Zephyr обеспечивает нативную поддержку протоколов, критически важных для IoT-устройств: MQTT, TLS. В-третьих, механизм MCUboot, интегрированный в экосистему Zephyr, обеспечивает безопасную загрузку и надёжное FOTA-обновление прошивки. В-четвёртых, активное сообщество и корпоративная поддержка со стороны Nordic, Intel и других участников гарантируют долгосрочное развитие проекта и своевременное устранение ошибок.

Таким образом, операционные системы реального времени представляют собой фундаментальный программный компонент современных встроенных систем, обеспечивающий детерминированное планирование задач, надёжную синхронизацию и структурированное управление ресурсами. Zephyr RTOS, сочетая строгие гарантии реального времени с богатой экосистемой сетевых протоколов, гибкой системой конфигурации и поддержкой платформы nRF9161, является оптимальным выбором для реализации практической части настоящей работы. В следующем разделе рассматриваются средства отладки и тестирования встроенных систем.

2.3 Средства отладки и тестирования встроенных систем

Отладка и тестирование программного обеспечения для встроенных систем представляют собой одни из наиболее трудоёмких и специфичных этапов разработки. В отличие от прикладного программирования, где разработчик располагает богатым набором инструментов отладки непосредственно на целевой машине, разработчик встроенных систем сталкивается с принципиально иными условиями: ограниченным или полностью отсутствующим пользовательским интерфейсом, невозможностью остановить систему без нарушения её функционирования в ряде сценариев, труднодоступностью целевого оборудования и зависимостью поведения системы от реального времени. Совокупность этих факторов обуславливает применение специализированного инструментария, охватывающего как аппаратные средства взаимодействия с целевой платформой, так и программные инструменты анализа и тестирования.

Аппаратные средства отладки

Основу аппаратной отладки встроенных систем составляют стандартизированные отладочные интерфейсы, обеспечивающие прямой доступ к внутреннему состоянию процессора - регистрам, памяти, периферии - без вмешательства в выполняемый программный код.

Интерфейс JTAG (Joint Test Action Group) является наиболее распространённым отладочным интерфейсом в индустрии встроенных систем. Первоначально разработанный для сканирования печатных плат с целью тестирования паяных соединений, JTAG впоследствии был адаптирован для внутрисхемной отладки (In-Circuit Debugging, ICD) процессоров. Интерфейс использует четыре обязательных сигнала - TCK (тактовый), TMS (выбор режима), TDI (вход данных), TDO (выход данных) - и опциональный сигнал TRST (сброс). Цепочка JTAG позволяет объединять несколько устройств последовательно, что особенно актуально при отладке многокристальных систем.

Интерфейс SWD (Serial Wire Debug) является двухпроводной альтернативой JTAG, разработанной ARM специально для микроконтроллеров семейства Cortex-M. Используя лишь два сигнала - SWDCLK и SWDIO - SWD обеспечивает функциональность, сопоставимую с JTAG, при значительно меньшем числе выводов. Это критично для корпусов с ограниченным

числом контактов, характерных для микроконтроллеров в промышленных применениях. nRF9161 поддерживает интерфейс SWD, реализованный на отладочной плате nRF9161-DK через встроенный программатор J-Link OB (On-Board).

Аппаратный отладчик, подключаемый к интерфейсу JTAG/SWD, обеспечивает следующие возможности: остановку выполнения программы в произвольной точке посредством аппаратных точек останова (hardware breakpoints), пошаговое выполнение на уровне инструкций или строк исходного кода, чтение и запись регистров процессора и областей памяти, просмотр и изменение значений переменных, а также программирование флеш-памяти целевого устройства.

Осциллографы и логические анализаторы

Аппаратные измерительные приборы занимают важное место в арсенале разработчика встроенных систем, позволяя наблюдать физические сигналы на выводах микроконтроллера и периферийных устройств - то, что недоступно программным средствам отладки.

Осциллограф отображает форму электрического сигнала во времени, позволяя измерять амплитуду, частоту, временные интервалы и фронты сигналов. При отладке встроенных систем осциллограф применяется для верификации временных диаграмм протоколов SPI и I²C, измерения длительности импульсов PWM, контроля целостности сигналов на высоких частотах, диагностики проблем питания (просадки напряжения при пиковом потреблении тока) и измерения реального времени выполнения критических участков кода посредством переключения GPIO в начале и конце измеряемого участка - техника, известная как «GPIO toggling».

Логический анализатор записывает цифровые сигналы на множестве каналов одновременно и декодирует их в соответствии с протоколами цифровой связи. Современные логические анализаторы поддерживают аппаратное декодирование протоколов UART, SPI, I²C, CAN, USB и многих других, отображая расшифрованные транзакции в читаемом виде. Это незаменимо при отладке взаимодействия микроконтроллера с внешними датчиками, дисплеями, модулями памяти и коммуникационными чипами.

Программные средства отладки

GNU Debugger (GDB) является стандартным отладчиком для проектов на базе GCC-цепочки инструментов, в том числе для встроенных систем ARM. В сочетании с GDB-сервером, предоставляемым аппаратным отладчиком (JLinkGDBServer, OpenOCD), GDB обеспечивает полноценную отладку на целевом оборудовании: установку программных и аппаратных точек останова, пошаговое выполнение, просмотр стека вызовов, чтение и изменение памяти и регистров. Интеграция GDB с современными IDE позволяет использовать графический интерфейс отладки вместо командной строки: расширение Cortex-Debug для VS Code обеспечивает полнофункциональную графическую отладку ARM Cortex-M устройств с поддержкой J-Link и OpenOCD.

Технология RTT (Real-Time Transfer), разработанная SEGGER, обеспечивает двусторонний обмен данными между устройством и хост-компьютером через отладочный интерфейс SWD/JTAG без использования дополнительных аппаратных интерфейсов и без остановки выполнения программы. В отличие от вывода отладочных сообщений через UART, RTT не требует выделения аппаратного UART и не вносит значимых задержек: запись сообщения в RTT-буфер занимает единицы микросекунд. Данные считываются J-Link в фоновом режиме через SWD, не влияя на выполнение программы. Это делает RTT предпочтительным механизмом отладочного вывода в системах с жесткими требованиями реального времени. Zephyr RTOS поддерживает RTT как один из бэкендов подсистемы логирования, активируемый конфигурационным параметром `CONFIG_LOG_BACKEND_RTT=y`.

Подсистема логирования Zephyr

Zephyr предоставляет развитую подсистему логирования, обеспечивающую структурированный вывод диагностических сообщений с указанием уровня серьезности, имени модуля и временной метки. Подсистема поддерживает четыре уровня логирования - ERR, WRN, INF, DBG - и позволяет настраивать уровень независимо для каждого модуля. Ключевой особенностью является режим отложенной обработки (deferred logging): сообщения буферизуются в кольцевом буфере и выводятся в выделенном потоке низкого приоритета, что минимизирует влияние логирования на временные характеристики критических путей выполнения:

```
#include <zephyr/logging/log.h>
LOG_MODULE_REGISTER(app_sensor, LOG_LEVEL_DBG);

void sensor_read(void)
{
    int val = read_adc();
    LOG_INF("ADC value: %d", val);
    if (val < THRESHOLD) {
        LOG_WRN("Value below threshold");
    }
}
```

Рисунок 6

Zephyr Twister

Twister является встроенным в Zephyr инструментом автоматизации тестирования, обеспечивающим запуск тестовых наборов как на реальном оборудовании, так и на симуляторах. Twister осуществляет обнаружение тестов в директории проекта по файлам `testcase.yaml`, сборку тестовых конфигураций для целевых платформ, захват вывода и анализ результатов. Интеграция Twister в CI-конвейер (GitHub Actions, GitLab CI) обеспечивает автоматическое регрессионное тестирование при каждом коммите.

Типовая структура теста в Zephyr с использованием фреймворка Ztest выглядит следующим образом:

```
#include <zephyr/ztest.h>

ZTEST_SUITE(sensor_suite, NULL, NULL, NULL, NULL, NULL);

ZTEST(sensor_suite, test_reading_valid_range)
{
    int val = sensor_read_mock(MOCK_VALID_INPUT);
    zassert_between_inclusive(val, 0, 4095,
        "ADC value out of range");
}

ZTEST(sensor_suite, test_reading_overflow)
{
    int ret = sensor_read_mock(MOCK_OVERFLOW_INPUT);
    zassert_equal(ret, -ERANGE,
        "Expected -ERANGE on overflow");
}
```

Рисунок 7

Глава 3. Разработка и реализация программного обеспечения для встроенной системы

3.1 Постановка задачи и выбор аппаратной платформы

В рамках практической части настоящей работы разрабатывается программное обеспечение для встроенного IoT-устройства, предназначенного для сбора данных с физических датчиков и их передачи в облачную инфраструктуру посредством протокола MQTT через сотовую сеть стандарта LTE-M. Устройство функционирует автономно, без постоянного присутствия обслуживающего персонала, и должно обеспечивать надёжную передачу данных, удалённое управление конфигурацией и обновление прошивки по беспроводному каналу (FOTA Upgrade).

Целевое применение устройства предполагает его установку в условиях промышленной эксплуатации, где физический доступ к оборудованию затруднён, а требования к надёжности функционирования и автономности работы являются приоритетными. В соответствии с данными требованиями к разрабатываемому программному обеспечению предъявляется следующий комплекс функциональных и нефункциональных требований.

Функциональные требования определяют поведение системы: периодический опрос подключённых датчиков (датчик вибрации, датчик расстояния) и аналого-цифровых каналов; формирование структурированных пакетов данных и их публикация на MQTT-брокер; подписка на управляющие топики MQTT для получения команд и обновлений конфигурации в реальном времени; проверка доступности обновлений прошивки и их загрузка посредством механизма FOTA; управление жизненным циклом сотового соединения с автоматическим восстановлением при разрыве.

Нефункциональные требования задают характеристики качества системы: детерминированное поведение планировщика задач с гарантированным временем реакции на события; минимизация энергопотребления за счёт применения режима PSM модема nRF9161; надёжность функционирования при нестабильном сотовом покрытии; возможность удалённого обновления прошивки без физического доступа к устройству; структурированное логирование для диагностики в процессе эксплуатации.

Выбор и обоснование аппаратной и прогаммной платформы

В качестве аппаратной платформы для реализации разрабатываемого устройства выбран модуль *Nordic Semiconductor nRF9161*, представляющий собой системный модуль (SiP, System-in-Package) с интегрированным процессорным ядром ARM Cortex-M33 и сертифицированным модемом стандарта LTE. Данный выбор обусловлен совокупностью технических и экосистемных факторов, детально рассматриваемых ниже.

Архитектура nRF9161. Процессорное ядро ARM Cortex-M33 с тактовой частотой до 64 МГц обеспечивает достаточную вычислительную мощность для реализации прикладной логики, обработки данных датчиков и управления сетевым стеком. Объём встроенной флеш-памяти составляет 1 МБ, оперативной памяти - 256 кБ, что является достаточным для размещения операционной системы Zephyr, прикладного кода и сетевых буферов.

Интегрированный модем LTE-M/NB-IoT обеспечивает подключение к сотовым сетям поколений Cat-M1 и Cat-NB2 в диапазонах частот, охватывающих большинство регионов мира. Модем поддерживает протокол энергосбережения PSM (Power Saving Mode) позволяющее радикально снизить среднее энергопотребление устройства в режиме периодической передачи данных. Встроенный аппаратный ускоритель криптографических операций и хранилище ключей обеспечивают поддержку TLS без значительной нагрузки на процессорное ядро.

Экосистема разработки. Nordic Semiconductor предоставляет для nRF9161 полноценный SDK - nRF Connect SDK - основанный на Zephyr RTOS и включающий все необходимые компоненты: драйвер модема `nrf_modem_lib`, библиотеку управления соединением LTE, сетевой стек с поддержкой MQTT и HTTPS, интеграцию с MCUboot для FOTA, а также обширный набор примеров приложений.

Отладочная платформа. Для разработки и тестирования используется отладочная плата nRF9161-DK, оснащённая встроенным программатором J-Link OB, интерфейсами GPIO, UART, SPI и I²C, а также слотом для SIM-карты. Наличие встроенного J-Link исключает необходимость внешнего программатора и обеспечивает полноценную отладку через SWD непосредственно из среды разработки.

В качестве операционной системы реального времени выбрана Zephyr RTOS версии, включённой в актуальный релиз nRF Connect SDK. Обоснование данного выбора детально представлено в разделе 2.2 данной работы; здесь лишь отметим ключевые факторы применительно к конкретной задаче:

нативная поддержка nRF9161 и модема LTE, встроенный MQTT-клиент, интеграция MCUboot для FOTA, система конфигурации Kconfig и Device Tree, обеспечивающие гибкое управление составом компонентов. Система сборки проекта основана на CMake и инструменте west, обеспечивающих воспроизводимую сборку с фиксированными версиями зависимостей через west manifest. Язык реализации - C++ стандарта C++ 17, что соответствует обоснованию, представленному в разделе 2.1.

3.2 Проектирование архитектуры программного обеспечения

Анализ требований и определение задач системы

В соответствии с этапом анализа требований, описанным в разделе 1.3, на начальной стадии проектирования были формализованы задачи (threads) системы, их временные характеристики и объекты синхронизации. Анализ функциональных требований привёл к выделению следующих логических подсистем: управление сотовым соединением и MQTT-сессией, периодический мониторинг датчиков и публикация данных, обработка входящих MQTT-сообщений, управление обновлением прошивки. Каждая из перечисленных подсистем реализована в виде отдельного программного компонента с чётко определённым интерфейсом и областью ответственности.

Программное обеспечение разрабатываемого устройства построено по многоуровневому принципу, соответствующему архитектурной модели, рассмотренной в разделе 1.2. Нижний уровень образуют драйверы периферии Zephyr - ADC, GPIO, UART - взаимодействующие с аппаратными модулями через HAL. Над ними располагается уровень абстракции датчиков, скрывающий детали аппаратного интерфейса за унифицированным API. Следующий уровень составляют менеджеры подсистем - MqttManager, ConnectionManager, UpgradeManager, RequestManager - реализующие протокольную логику и управление состоянием. Верхний уровень образует компонент SystemMonitor, реализующий главный поток обработки и координирующий взаимодействие всех подсистем.

Иерархия классов и компонентов

Иерархия датчиков построена на принципе наследования с общим базовым классом `Sensor`, определяющим универсальный интерфейс опроса устройства. Базовый класс содержит атрибуты идентификации (`version`, `name`) и управления питанием (`powerOn`, `powerOff`). От `Sensor` наследуются конкретные реализации: `SensorVibration`, инкапсулирующий логику работы с датчиком вибрации и предоставляющий методы `readRaw()`, `readRMS()`, `powerOn()`, `powerOff()`; и `SensorDistance`, реализующий работу с датчиком расстояния с аналогичным набором методов. Данная архитектура обеспечивает единообразную обработку разнородных датчиков в коде верхнего уровня и упрощает добавление новых типов датчиков без модификации существующего кода.

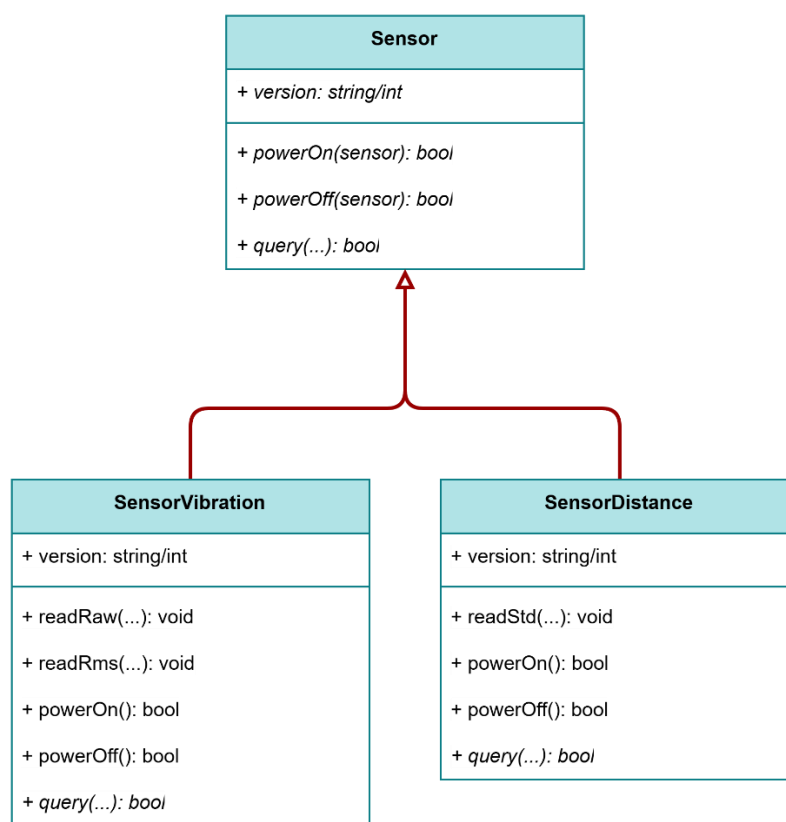


Рисунок 8

Компонент `Adc` инкапсулирует работу с аналого-цифровым преобразователем `nRF9161` через подсистему `Zephyr ADC`. Компонент хранит статическую таблицу конфигурации каналов (`m_pTable`) и предоставляет методы инициализации (`init()`), добавления каналов (`addAdcConfig()`), считывания значений (`readRaw()`, `readMm()`) и установки статуса выводов (`setStatus()`). Использование статической таблицы каналов исключает динамическое выделение памяти и обеспечивает предсказуемое потребление ОЗУ.

Adc
- <code>AdcConfig *m_pTable</code>
+ <code>instance(): Adc&</code>
+ <code>init(AdcConfig *pTable): bool</code>
+ <code>readRaw(uint8_t id, int *pVal): bool</code>
+ <code>readMv(uint8_t id, int *pVal): bool</code>
+ <code>setStatus(id, PinStatus status): bool</code>

Рисунок 9

Компонент `Gpio` обеспечивает управление цифровыми выводами микроконтроллера через подсистему `Zephyr GPIO`. Аналогично `Adc`, хранит статическую таблицу конфигурации выводов (`m_pTable`) и предоставляет методы инициализации, установки значений и считывания состояния.

Gpio
- <code>GpioConfig *m_pTable</code>
+ <code>instance(): Gpio&</code>
+ <code>init(GpioConfig *pTable): bool</code>
+ <code>set(uint8_t id, int val): bool</code>
+ <code>get(uint8_t id, int *pVal): bool</code>
+ <code>setStatus(id, PinStatus status): bool</code>

Рисунок 10

Компонент RequestManager реализует обработку запросов с помощью публичных методов sendOnDemandResp(), sendStdReport(), sendRawReport(), sendRmsReport().

Компонент HTTP инкапсулирует логику HTTP-запросов, используемых для загрузки обновлений прошивки.

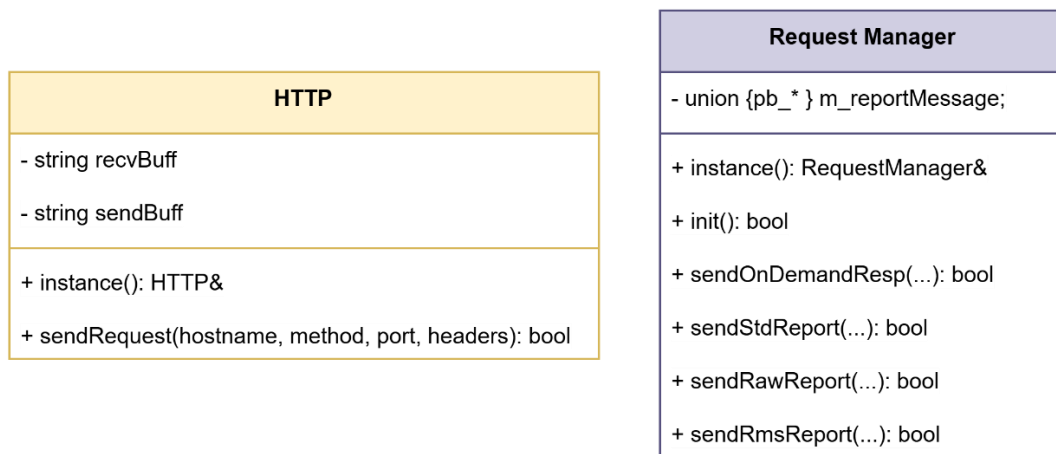


Рисунок 11

Компонент MqttManager

MqttManager является центральным компонентом уровня сетевого взаимодействия. Он хранит конфигурационные параметры подключения (topics) и очередь событий (eventQueue), а также предоставляет полный набор методов управления MQTT-сессией: connect(), disconnect(), publish(), subscribe(), onConnect(), onDisconnect(), onMessage(), addSubscription(), checkConnection().

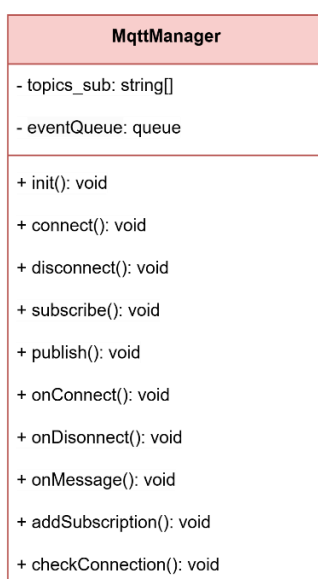


Рисунок 12

Функция `publish()` реализует публикацию сообщений на MQTT-брокер. Перед вызовом публикации проверяется состояние соединения посредством флага `MQTT_CON`; при отсутствии активного соединения сообщение помещается в очередь для последующей отправки. Данный подход обеспечивает устойчивость к временным разрывам соединения без потери данных датчиков. Функция `subscribe()` регистрирует подписки на управляющие MQTT-топики и сохраняет соответствие топики и обработчиков в словаре подписок (`addSubscription()`).

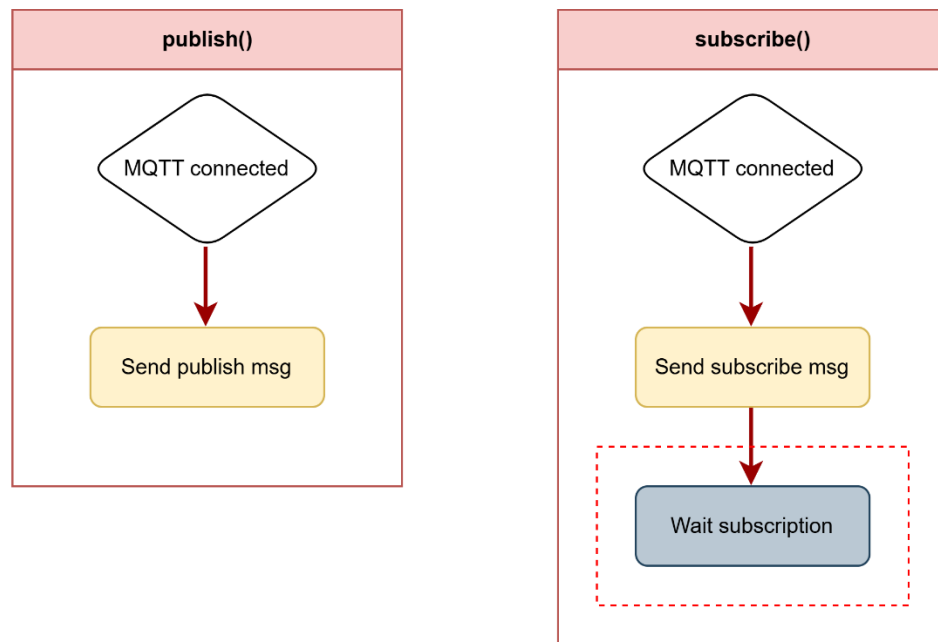


Рисунок 13

Компонент UpgradeManager

UpgradeManager инкапсулирует логику управления процессом обновления прошивки. Компонент предоставляет методы инициализации (`init()`), проверки необходимости обновления (`checkAndUpgradeDevice()`) и выполнения обновления `selfUpgrade`. Метод `checkAndUpgrade()` в контексте `SystemMonitor` реализует следующую логику: если системная версия (`self_version`) меньше версии, доступной на файловом сервере (`file_version`), инициируется `selfUpgrade(file)`. Атомарность процесса обновления обеспечивается механизмом MCUboot с поддержкой отката, рассмотренным в разделе 1.3.

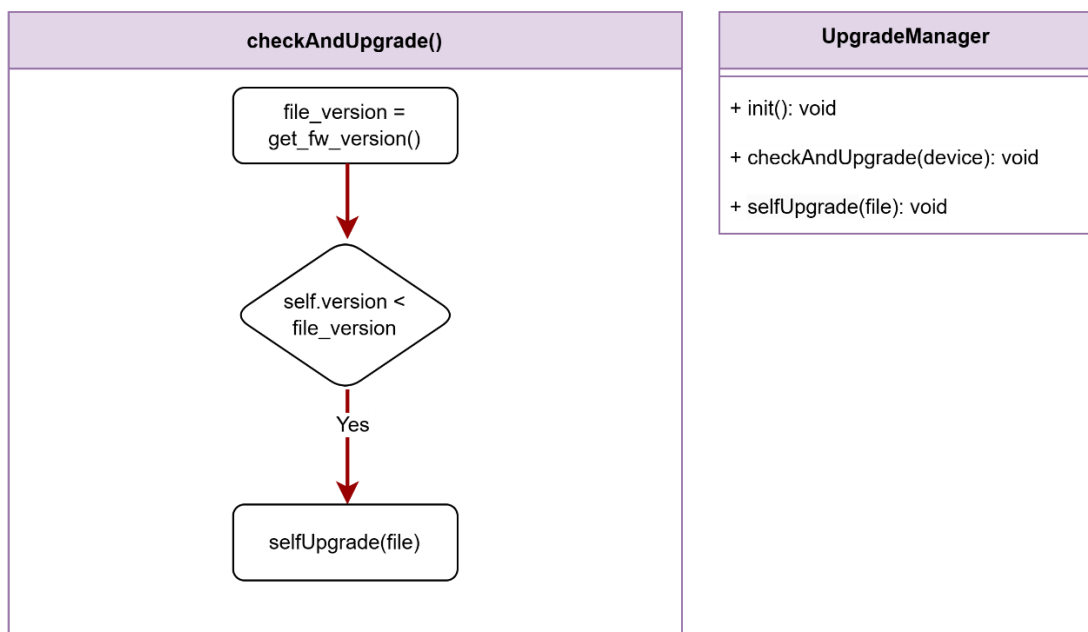


Рисунок 14

Конечный автомат ConnectionManager

Наиболее архитектурно значимым компонентом системы является `ConnectionManager`, реализующий управление жизненным циклом сотового соединения и MQTT-сессии в виде иерархического конечного автомата. Применение паттерна FSM обусловлено сложностью и нелинейностью процесса установки соединения в сотовых сетях: регистрация в сети, установка PDP-контекста, подключение к MQTT-брокеру и поддержание активности сессии представляют собой последовательность асинхронных событий с возможностью отказа на любом этапе.

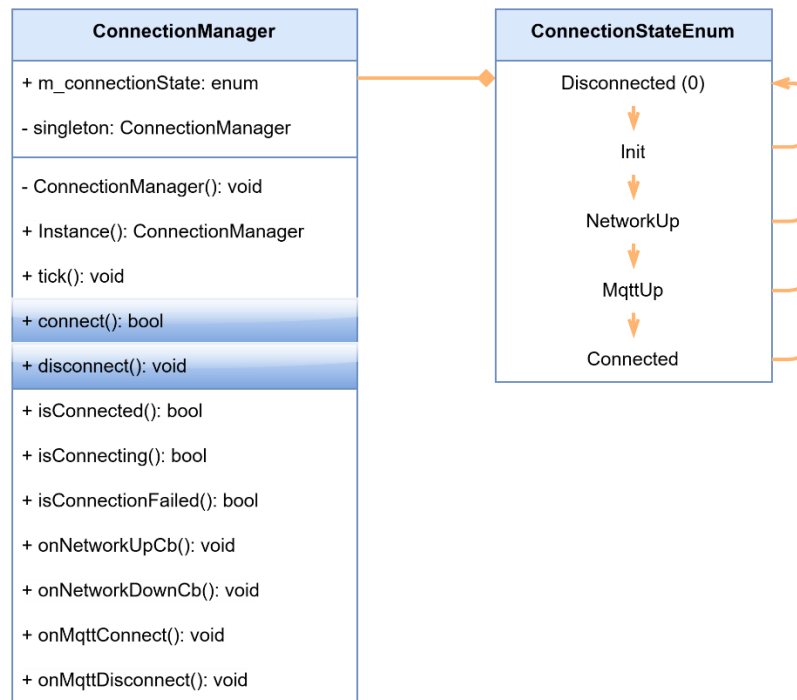


Рисунок 15

Автомат определяет следующий набор состояний, формализованных перечислением ConnectionStateEnum.

- Состояние Init является начальным состоянием после инициализации компонента. В данном состоянии осуществляется инициализация модема (init()), проверка доступности сети и ожидание события Network Up. При получении события сеть доступна автомат переходит в состояние NetworkUp; в противном случае устанавливается состояние NetworkUp с флагом ожидания.
- Состояние NetworkUp соответствует успешной регистрации устройства в сотовой сети. В данном состоянии проверяется флаг MQTT_CON: если MQTT-соединение уже установлено, автомат немедленно переходит в состояние MqttUp; в противном случае инициируется процедура подключения к MQTT-брокеру (MQTT Connect) и устанавливается состояние MqttUp.
- Состояние MqttUp соответствует успешному установлению TCP-соединения с MQTT-брокером и завершению MQTT-рукопожатия. В данном состоянии отправляется событие Send MQTT Connected Event и устанавливается состояние Connected.

- Состояние Connected является штатным рабочим состоянием системы, в котором осуществляется периодическая публикация данных датчиков и обработка входящих сообщений. При получении события On Network Down Callback (потеря сотового покрытия) сбрасывается флаг NET_CON, выполняется MQTT Disconnect и устанавливается состояние Disconnected. При получении события On MQTT Disconnected Callback (разрыв MQTT-сессии без потери сети) сбрасывается флаг MQTT_CON и устанавливается состояние NetworkUp для повторного подключения к брокеру.
- Состояние Disconnected соответствует полной потере сетевого соединения. В данном состоянии система ожидает события On Network Up Callback, после получения которого устанавливается флаг NET_CON и автомат переходит в состояние NetworkUp для повторного прохождения процедуры подключения.

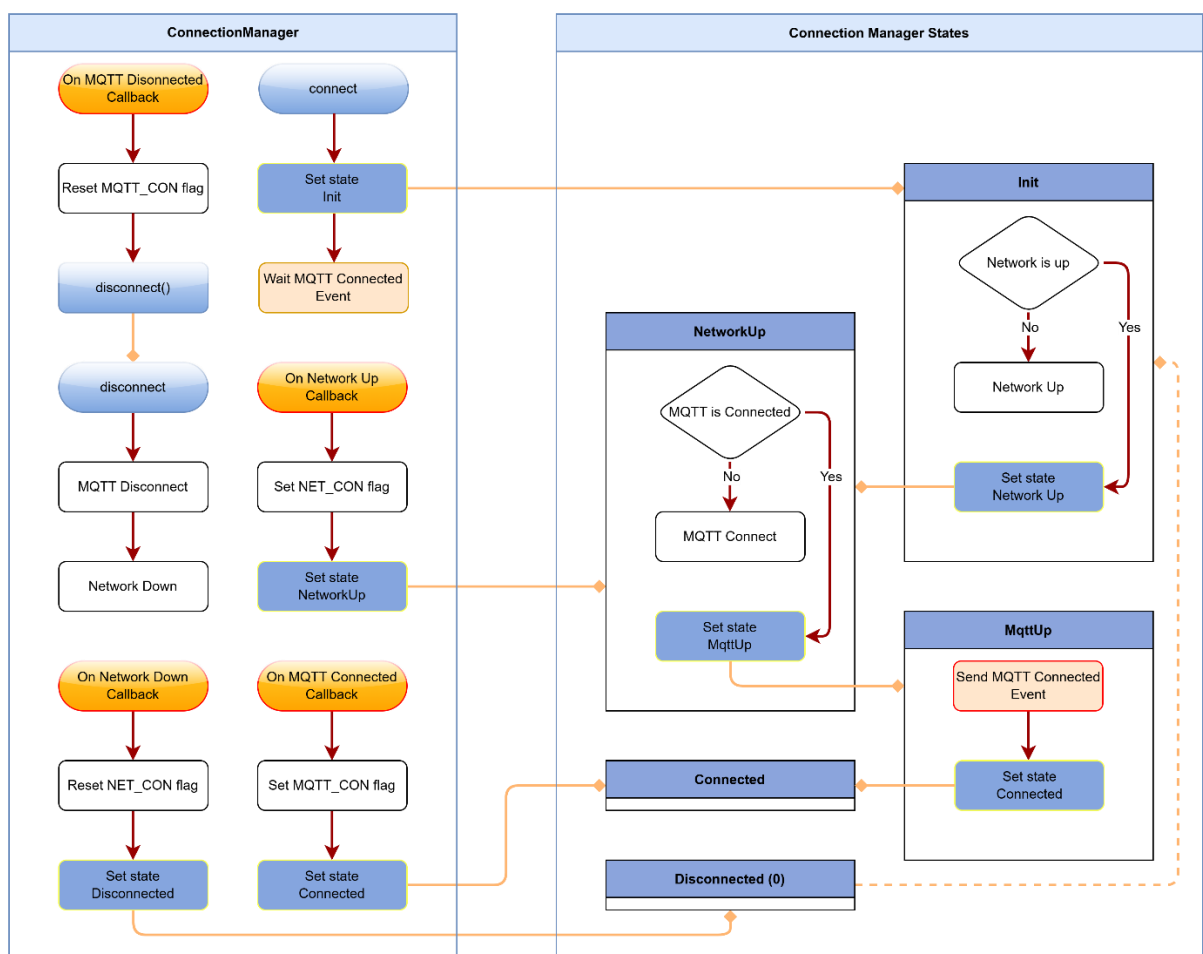


Рисунок 16. Конечная машина ConnectionManager

Применение конечного автомата обеспечивает ряд важных свойств системы. Во-первых, полную обработку всех возможных сценариев разрыва и восстановления соединения без дублирования кода. Во-вторых, чёткую прослеживаемость поведения системы - каждое состояние и переход задокументированы и верифицируемы. В-третьих, устойчивость к нестабильному сотовому покрытию - система автоматически восстанавливает соединение без вмешательства пользователя.

3.3 Реализация и тестирование программного обеспечения

Проект реализован на базе чипа Nordic Semiconductor nRF9161 и организован в соответствии с расширенной структурой приложения Zephyr с явным разделением по функциональным слоям. Корневой каталог содержит файлы CMakeLists.txt, CMakePresets.json, Kconfig, sysbuild.conf и каталог boards/nordic/ с пользовательским описанием платы в формате Device Tree. Наличие CMakePresets.json обеспечивает именованные конфигурации сборки для различных аппаратных вариантов платы, что упрощает воспроизводимость сборочного окружения в командной разработке.

Исходный код организован по следующим каталогам: Application/ - прикладная логика верхнего уровня;

- Core/ - ключевые системные компоненты
- BSP/ (Board Support Package) - поддержка конкретной платы
- Drivers/ - драйверы периферийных устройств; Libraries/ - вспомогательные библиотеки
- Net/ - сетевой уровень и MQTT
- Sensor/ - реализация иерархии датчиков
- Utils/ - утилитарные компоненты
- Tests/ - тестовая инфраструктура

Инструментарий разработки основан на nRF Connect SDK версии 3.1.0 с расширением nRF Connect for VS Code, обеспечивающим интеграцию сборки, прошивки и отладки непосредственно в среде Visual Studio Code.

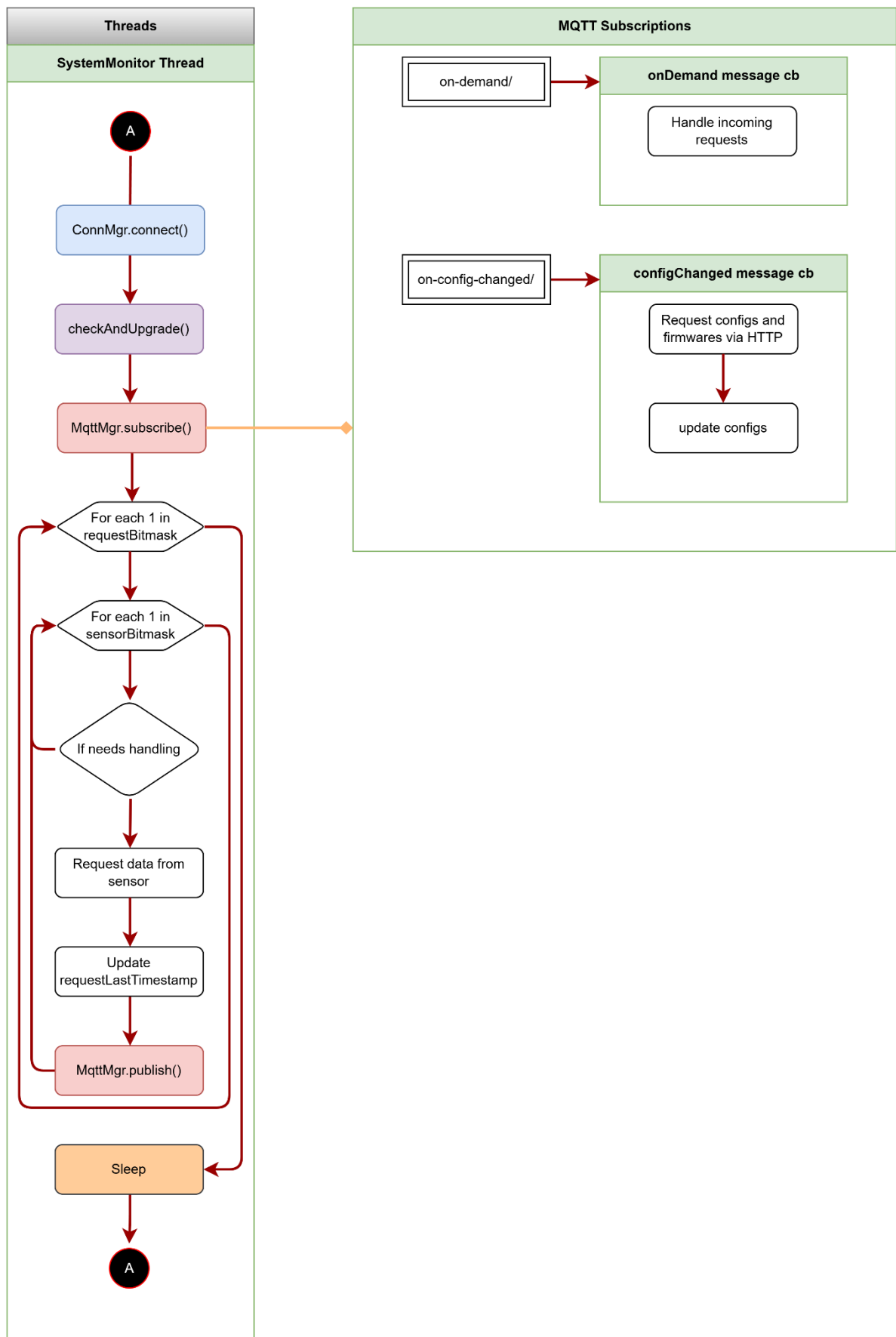


Рисунок 17. Основные потоки управления

На рисунке 17 представлена диаграмма потоков управления главного потока SystemMonitor и подсистемы MQTT-подписок, иллюстрирующая взаимодействие между циклической логикой основного потока и событийно-управляемой обработкой входящих MQTT-сообщений.

Главный поток SystemMonitor Thread реализован в виде бесконечного цикла, обозначенного соединителями А в начале и конце диаграммы. На каждой итерации цикла последовательно выполняются следующие шаги.

Первым вызывается ConnMgr.connect() - установка или верификация активности сотового соединения и MQTT-сессии посредством конечного автомата ConnectionManager.

Вторым шагом выполняется checkAndUpgrade() - проверка доступности обновления прошивки и его применение при необходимости. Третьим шагом вызывается MqttMgr.subscribe(), регистрирующий активные подписки на MQTT-топики. Именно в этой точке главный поток взаимодействует с подсистемой MQTT-подписок - связь обозначена стрелкой с ромбом на диаграмме.

После установки подписок выполняется двухуровневый вложенный цикл обработки запросов датчиков. Внешний цикл итерирует по битовой маске запросов requestBitmask, внутренний - по битовой маске активных датчиков sensorBitmask. Если датчик требует обработки в данной итерации - в соответствии с его временным интервалом - выполняются последовательно запрос данных с датчика (Request data from sensor), обновление временной метки последнего запроса (Update requestLastTimestamp) и публикация накопленных данных на MQTT-брокер (MqttMgr.publish()).

Если датчик не требует обработки в текущей итерации, внутренний цикл переходит к следующему элементу sensorBitmask. По завершении обхода всех датчиков поток переходит в режим сна (Sleep) на интервал до следующего запланированного события, после чего цикл начинается заново с соединителя А.

Правая часть диаграммы представляет подсистему MQTT-подписок, функционирующую независимо от главного потока в событийно-управляемом режиме. Определены два топика с соответствующими функциями обратного вызова. Топик on-demand/ обрабатывается колбэком onDemand message cb, который принимает входящий запрос и передаёт его на обработку - Handle incoming requests - в OnDemandHandler, формирующий внеплановый опрос датчиков вне основного цикла. Топик on-

config-changed/ обрабатывается колбэком configChanged message cb, выполняющим загрузку обновлённой конфигурации и прошивки через HTTPS (Request configs and firmwares via HTTP) с последующим применением новой конфигурации (update configs).

Диаграмма наглядно отражает ключевое архитектурное решение системы: разделение потоков управления на детерминированный периодический цикл сбора данных и асинхронную событийно-управляемую обработку входящих MQTT-команд. Данное разделение обеспечивает предсказуемость временного поведения основного цикла при сохранении возможности оперативной реакции на внешние управляющие воздействия без нарушения запланированных интервалов опроса датчиков.

Компонент SysCtl и точка входа приложения

Точка входа приложения реализована в функции main() и представляет собой строго упорядоченную последовательность инициализации системных компонентов с явной проверкой результата каждого шага. Данный подход соответствует рекомендациям по разработке надёжного встроенного ПО: любой сбой инициализации критически важного компонента немедленно прерывает запуск с возвратом кода ошибки, исключая запуск системы в частично инициализированном состоянии.

Первым действием после входа в main() активируется подсистема логирования посредством LogController::instance()->enableAppLogs(), что обеспечивает доступность диагностического вывода на всех последующих этапах инициализации. Немедленно после этого выводится версия прошивки - мажорная, минорная и патч-версии, извлекаемые через макросы versionGetMajor(), versionGetMinor(), versionGetPatch().

Принципиально важным следующим шагом является вызов boot_write_img_confirmed() из подсистемы MCUboot. Данный вызов подтверждает корректность текущего образа прошивки, переводя его из состояния «тестового» в состояние «подтверждённого». Если подтверждение не происходит в течение заданного тайм-аута, MCUboot при следующей перезагрузке автоматически выполняет откат к предыдущей версии прошивки. Размещение этого вызова в самом начале main() - до любой прикладной логики - гарантирует, что прошивка, не способная даже инициализироваться, будет откатана автоматически.

При активированном конфигурационном параметре CONFIG_RTT_LOG перед запуском основного цикла выполняется пятисекундная задержка с обратным отсчётом, выводимым через RTT-бэкенд логирования. Это предоставляет разработчику временное окно для подключения клиента RTT

(SEGGER J-Link RTT Viewer или JLinkRTTLogger) до начала выполнения прикладного кода - практически значимое удобство при отладке процедуры инициализации.

Далее инициализируется аппаратный сторожевой таймер (Watchdog) с таймаутом CONFIG_WATCHDOG_DEFAULT_TIMEOUT_MS, определяемым через Kconfig. Сторожевой таймер обеспечивает автоматическую перезагрузку устройства при зависании любого компонента системы - критически важная мера защиты для автономного IoT-устройства, не имеющего возможности ручного перезапуска в условиях эксплуатации.

После инициализации сторожевого таймера выполняется инициализация подсистемы энергонезависимого хранилища NVS (Non-Volatile Storage) - Flash-хранилища Zephyr для персистентных параметров конфигурации. Неуспешная инициализация NVS является фатальной ошибкой: без доступа к конфигурации устройство не может корректно функционировать.

Завершает фазу инициализации вызов SysCtl::instance().setup(), делегирующий полную инициализацию всех прикладных подсистем компоненту SysCtl. После успешного завершения setup() управление передаётся основному циклу:

```
while (true) {
    ret = SysCtl::instance().mainloop();
    if (!ret) {
        LOG_ERR("System controller run failed, retrying in 300ms...");
        k_msleep(300);
    }
    k_msleep(10);
}
```

Рисунок 18

Структура основного цикла отражает важное архитектурное решение: mainloop() является возвращаемой функцией, а не бесконечным циклом внутри. При возврате false система выдерживает паузу 300 мс и повторяет попытку. Это обеспечивает устойчивость к временным сбоям сетевого соединения без перезагрузки устройства, тогда как зависание в любой точке mainloop() будет обнаружено сторожевым таймером.

Компонент SysCtl

SysCtl (System Controller) является центральным управляющим компонентом приложения, реализованным как потокобезопасный синглтон с

запрещёнными операциями копирования. Компонент координирует все прикладные подсистемы устройства - управление соединением, сбор данных датчиков, обработку конфигурации, механизм обновления прошивки и управление энергопотреблением.

Состояние компонента включает четыре временные метки следующих событий - `m_nextStd`, `m_nextRms`, `m_nextRaw`, `m_nextGnss` - хранящих абсолютное время в миллисекундах (`uptime`) момента, когда соответствующий тип сбора данных должен быть выполнен в следующий раз. Данный подход реализует таймерную логику без использования RTOS-таймеров: вместо регистрации колбэков по таймерам компонент на каждой итерации `mainloop()` сравнивает текущее время `Timer::uptimeMs()` с запланированными дедлайнами и выполняет соответствующие операции.

Инициализация: `setupDevice` и `setupNetwork`

Метод `setup()` разделяет инициализацию на два этапа, отражающих логическое разделение аппаратного и сетевого уровней.

`setupDevice()` инициализирует аппаратно-зависимые компоненты: BSP (Board Support Package) через `BSP::init()`, компонент `DeviceInfo` - хранилище идентификационных данных устройства (IMEI, серийный номер, аппаратная ревизия) - и `RequestManager`, ответственный за формирование и отправку отчётов. После инициализации устройства проверяется наличие SIM-карты посредством `ConnectionManager::isSimPresent()`; при её отсутствии запускается визуальная индикация через `blinkForSim()` - практически значимая функция для производственного тестирования.

`setupNetwork()` инициализирует сетевой стек: `ConnectionManager` с передачей IMEI как идентификатора устройства, `MqttManager`, драйвер модема `Modem` и, при активированной опции `CONFIG_GNSS`, менеджер геолокации `GnssManager`. После завершения `setupNetwork()` регистрируются два MQTT-топика с соответствующими колбэками: топик конфигурации (`m_onConfigChangedTopic`, формируемый макросом `MQTT_CONFIG_CHANGED_TOPIC` с подстановкой IMEI) и топик on-demand запросов (`m_onDemandTopic`).

Основной цикл mainloop

Метод mainloop() реализует полный цикл работы устройства - от установки соединения до перехода в режим сна. Структура метода отражает приоритизацию операций и политику энергосбережения.

На первом этапе устанавливается сетевое соединение посредством ConnectionManager::instance().connect() с тайм-аутом, задаваемым конфигурацией. При неудаче реализуется стратегия отступа: после NETWORK_CONNECTION_MAX_TRIALS = 5 последовательных неудачных попыток система переходит в режим сна на backoffTimeoutMs миллисекунд, определяемых конфигурацией с сервера. Счётчик попыток сбрасывается при каждом успешном подключении.

После первого успешного подключения после перезагрузки - условие m_isAfterReset - выполняется однократная операция: считывается и очищается регистр RESETREAS (Reset Reason Register) микроконтроллера по прямому адресу 0x40005000 + 0x400 и отправляется отчёт sendBatchDeviceCfg() с причиной перезагрузки. Очистка регистра производится записью 0xFFFFFFFF - в соответствии со спецификацией nRF9161, где поля регистра очищаются записью единицы. Флаг m_isAfterReset сбрасывается сразу после, гарантируя однократность отправки.

При наличии непрочитанных системных логов в EEPROM (MiLogger::instance()->getSysLogLength() > 0) перед основной обработкой выполняется отправка лог-отчёта sendLogMessageReport(). Данный механизм обеспечивает доставку диагностической информации, накопленной в EEPROM во время предыдущих сессий или в периоды отсутствия связи, на сервер при первой возможности после восстановления соединения.

Затем загружается актуальная конфигурация с сервера посредством DeviceInfo::instance().configGet(). При неудаче загрузки проверяется валидность ранее сохранённой конфигурации - при её отсутствии применяются значения по умолчанию configSetDefaults(). Этот паттерн обеспечивает работоспособность устройства даже при длительной недоступности сервера конфигурации.

После загрузки конфигурации выполняется checkAndUpgrade() - проверка наличия обновления прошивки. Метод проверяет флаг ltenodeUpdatesEnabled из конфигурации и сравнивает текущую версию с

версией, закодированной в имени файла прошивки на сервере, посредством `getVersionFromFilename()`. Дополнительно проверяется соответствие файла прошивки аппаратной ревизии платы (`REV_A_B` или `REV_C`), исключая установку несовместимого образа. При необходимости обновления вызывается `upgradeImage()`, реализующий четырёхэтапный процесс: `setup()` → `prepare()` → `start()` → `complete()`.

Сбор данных датчиков организован по типам с независимыми временными интервалами. Стандартные отчёты (`m_nextStd`) собираются с периодом `totalWindowSizeMs` и включают статистический отчёт `sendStatsReport()`, стандартные показания датчиков `takeStandardReport()` и отправку агрегированного отчёта `sendStdReport()`. Отчёты RMS и RAW вибрации с периодами `rmsCycle` и `rawCycle` обрабатываются совместно через `OnDemandHandler::instance().handleReadingRequest()`, принимающий комбинированную битовую маску типов измерений.

При активированной поддержке GNSS выполняется периодический сбор данных геолокации `GnssManager::instance().collectPvtFix()` с периодом `gnssInterval`.

Завершает итерацию `mainloop()` вызов `SysCtl::sleep()` - перехода в энергосберегающий режим на интервал до следующего запланированного события.

Механизм сна и событийное пробуждение

Метод `sleep()` реализует управление энергосбережением на основе вычисления ближайшего запланированного события. Функция `getNextEventMs()` определяет минимум среди всех запланированных временных меток (`m_nextStd`, `m_nextRaw`, `m_nextRms`, `m_nextGnss`), вычисляя тем самым оптимальную продолжительность сна.

Ожидание реализовано через `zbus_sub_wait()` - ожидание события на шине сообщений Zephyr ZBus с тайм-аутом, равным вычисленной длительности сна. Применение ZBus вместо простого `k_msleep()` обеспечивает возможность досрочного пробуждения при получении внешних событий, в частности нажатия физической кнопки (`BUTTON_TRIGGERED_EVENT`). При пробуждении по событию кнопки вызывается `Button::instance().handlePress()` для обработки нажатия, после чего цикл сна продолжается до наступления следующего запланированного события. Перед входом в режим сна и после пробуждения вызываются `BSP::preSleepSequence()` и `BSP::postWakeSequence()` соответственно -

функции управления периферией платы, обеспечивающие корректное отключение и восстановление аппаратных компонентов при переходе между режимами энергопотребления.

Таким образом, связка `main()` и `SysCtl` реализует полный жизненный цикл IoT-устройства: от строго упорядоченной инициализации с верификацией каждого компонента через событийно-управляемый основной цикл с независимыми временными интервалами сбора данных до энергосберегающего сна с возможностью досрочного пробуждения по внешним событиям.

Сериализация данных посредством Protocol Buffers

Принципиальным архитектурным решением, отличающим данный проект от типовых IoT-реализаций с JSON-сериализацией, является применение Protocol Buffers (protobuf) для формирования MQTT-сообщений.

Данное решение обусловлено рядом факторов, принципиально важных для IoT-устройств с сотовым соединением. Во-первых, бинарное представление данных в protobuf значительно компактнее JSON: типичное сообщение с данными датчиков занимает меньше байт, что напрямую снижает объем передаваемых данных и, следовательно, стоимость сотового трафика и энергопотребление радиомодуля. Во-вторых, схемы protobuf обеспечивают строгую типизацию и версионирование формата сообщений, что критично при долгосрочной эксплуатации устройств с различными версиями прошивки. В-третьих, кодирование и декодирование protobuf выполняется значительно быстрее парсинга JSON, что снижает нагрузку на процессорное ядро.

Для генерации C/C++-кода из .proto-схем применяется компилятор protoc с плагином nanorb - реализацией Protocol Buffers, специально адаптированной для встроенных систем с ограниченной памятью. nanorb генерирует код без динамической аллокации, используя статически выделенные структуры с фиксированными максимальными размерами полей, что соответствует принципам безопасного программирования для встроенных систем.

Реализация пользовательского бэкенда логирования MiLogger

Одним из наиболее архитектурно значимых компонентов разработанной системы является MiLogger - пользовательский бэкенд подсистемы логирования Zephyr, обеспечивающий персистентное хранение лог-сообщений в энергонезависимой памяти EEPROM типа FM24W256. Данное решение принципиально отличается от стандартных бэкендов Zephyr (UART, RTT): в то время как стандартные бэкенды обеспечивают вывод логов исключительно в режиме онлайн-соединения с отладочным терминалом, MiLogger сохраняет лог-сообщения в энергонезависимой памяти устройства, что позволяет извлекать диагностическую информацию постфактум - критически важное свойство для IoT-устройств, эксплуатируемых в труднодоступных местах без постоянного подключения отладчика. Архитектура и организация памяти EEPROM Память EEPROM FM24W256 объёмом 32 кБ разделена на три логических раздела с фиксированными границами, определяемыми макросами препроцессора. В начале памяти по адресу MILOGGER_HEADER_ADDRESS = 0x0 располагается заголовок (MiLogger::header) - управляющая структура фиксированного размера, хранящая метаданные состояния всей подсистемы логирования. Непосредственно за заголовком следует раздел системных логов (sysLog) объёмом в одну четверть доступной памяти за вычетом размера заголовка, предназначенный для хранения сообщений уровней WRN, ERR и INF. Оставшиеся три четверти памяти отведены под раздел отладочных логов (dbgLog), реализованный как кольцевой буфер и предназначенный для сообщений уровня DBG. Структура заголовка определена с атрибутом `__attribute__((packed))` для исключения выравнивания компилятором и обеспечения предсказуемого размещения в памяти:

```
struct __attribute__((packed)) header
{
    uint32_t sysLogHead;
    uint32_t sysLogTail;
    uint32_t dbgHead;
    uint32_t dbgTail;
    uint32_t flags;
    uint32_t logCount;
    uint32_t logCRC;
    uint32_t headerCRC;
};
```

Рисунок 19

Двойная схема CRC - отдельно для данных логов (logCRC) и для самого заголовка (headerCRC) - обеспечивает возможность детектирования повреждения как заголовка, так и данных при аномальном завершении записи, например вследствие внезапного отключения питания. Каждый отдельный лог-записи предшествует заголовок logHeader, также упакованный:

```
struct __attribute__((packed)) logHeader
{
    uint8_t  isValid;
    uint8_t  severity;
    uint32_t id;
    uint16_t subsystem;
    uint16_t messageLength;
    uint64_t timestamp;
};
```

Рисунок 20

Поле timestamp заполняется вызовом Timer::unixOrUptimeMs() - функции, возвращающей Unix-время при наличии синхронизации с сетью и время работы с момента загрузки в противном случае. Это обеспечивает осмысленную временную метку во всех режимах работы устройства.

Реализация паттерна Singleton и инициализация

MiLogger реализован как потокобезопасный синглтон с инициализацией при первом обращении посредством статической локальной переменной в методе instance(). Данный подход гарантирует единственность экземпляра в рамках всего приложения и корректную инициализацию без явного управления временем создания объекта.

Инициализация компонента выполняется методом init(), принимающим указатель на уже инициализированный объект драйвера FM24W256. В процессе инициализации создаются два мьютекса Zephyr - m_mutex для защиты доступа к EEPROM и m_eepromFlushMutex для защиты флага сброса буфера - записывается начальный заголовок с нулевыми счётчиками и вычисленным CRC, после чего посредством k_thread_create() создаётся выделенный поток MiLoggerPoll с размером стека 2048 байт и приоритетом 5, а бэкэнд разблокируется вызовом unlockBackend().

Двухуровневая буферизация и механизм отложенного сброса

Ключевым архитектурным решением MiLogger является двухуровневая буферизация, обеспечивающая минимальное влияние на временные характеристики системы при записи в медленную EEPROM-память.

Первый уровень - оперативный буфер в ОЗУ: двумерный массив `m_tempBuffer[32][MILOGGER_TMP_BUFFER_LEN]`, объявленный как `volatile` для корректного взаимодействия между потоком бэкенда и потоком сброса. При получении каждого лог-сообщения в методе `backendProcess()` текстовая часть формируется непосредственно в соответствующей строке буфера через коллбек `cbprintf_collect_cb`, после чего в начало строки копируется заполненный заголовок `logHeader`. Операция занимает единицы микросекунд и не блокирует вызывающий поток.

Второй уровень - периодический сброс в EEPROM: поток `MiLoggerPoll`, выполняющийся в бесконечном цикле с интервалом 100 мс, проверяет два условия активации сброса. Первое - достижение максимального индекса буфера (`m_tempBufferIdx == MILOGGER_TMP_BUFFER_IDX_MAX`, то есть накоплено 32 сообщения) - устанавливает флаг `m_eepromFlushTrigger`. Второе - истечение тайм-аута 2 секунды с момента последнего сброса при наличии хотя бы одного сообщения в буфере - гарантирует своевременную запись даже при низкой интенсивности логирования. При активации сброса поток последовательно записывает все накопленные буферизованные сообщения в EEPROM посредством `MiLogger::write()`.

Перед записью в EEPROM активируется `ScopedSenPwrController` и `ScopedAuxPwrController` - RAII-объекты управления питанием, обеспечивающие подачу питания на периферийные шины на время операции записи и автоматически отключающие его по завершении.

Маршрутизация логов по уровням серьезности

Статический метод `MiLogger::write()` реализует маршрутизацию каждого лог-сообщения в соответствующий раздел EEPROM в зависимости от уровня серьезности. Сообщения уровня `DBG` направляются в кольцевой раздел `dbgLog`; сообщения уровней `WRN` и `ERR` - в линейный раздел `sysLog`; сообщения уровня `INF` направляются в `sysLog` только при условии, что текущий уровень логирования установлен в `LOG_LEVEL_INF`. Данное разделение отражает различную семантику двух разделов: `sysLog` хранит критически важные события, не подлежащие вытеснению новыми записями (при переполнении устанавливается флаг `m_sysLogOverrunStatus`), тогда как `dbgLog` реализует политику кольцевого буфера - при нехватке места старые отладочные записи вытесняются новыми.

Механизм блокировки бэкенда и предотвращение рекурсии

Потенциальной проблемой при реализации пользовательского бэкенда логирования является рекурсия: если сам код бэкенда использует макросы LOG_*, их обработка вновь вызовет backendProcess(), что приведёт к бесконечной рекурсии или повреждению буфера. MiLogger решает данную проблему двумя взаимодополняющими механизмами.

Флаг m_backendIgnore типа utility::atomic_bool устанавливается в начале процедуры сброса в EEPROM через LogController::instance()->backendIgnore() и сбрасывается по её завершении через backendResume() с использованием utility::scope_exit - RAII-обёртки, гарантирующей выполнение операции сброса даже при досрочном выходе из функции. При установленном флаге backendProcess() немедленно возвращает управление без обработки сообщения.

Класс ScopedBackendLock реализует RAII-блокировку бэкенда для операций чтения и записи EEPROM: конструктор вызывает lockBackend() (отключение бэкенда через log_backend_disable()), деструктор - условно unlockBackend() в зависимости от предшествующего состояния блокировки. Это исключает поступление новых лог-сообщений в бэкенд во время выполнения операций с EEPROM.

Дополнительно, при установленном флаге m_eepromFlushTrigger (активный сброс в процессе) метод backendProcess() не буферизует новое сообщение, а инкрементирует счётчик m_droppedCount. По завершении сброса количество потерянных сообщений фиксируется в EEPROM отдельной синтетической записью вида "N Messages dropped", обеспечивая полную аудитируемость потерь при интенсивном логировании.

Регистрация бэкенда в подсистеме Zephyr

Регистрация MiLogger как бэкенда подсистемы логирования Zephyr осуществляется посредством макроса LOG_BACKEND_DEFINE, принимающего имя бэкенда, структуру log_backend_api с указателями на функции обработки и флаг автозапуска:

```
static const struct log_backend_api logBackendMiLoggerApi = {
    .process = logBackendMiLoggerProcess,
    .panic = [](const struct log_backend *const backend) { /* do nothing */ },
    .init = logBackendMiLoggerInit,
};

#if IS_ENABLED(CONFIG_EEPROM_LOGS)
constexpr bool autostart = true;
#else
constexpr bool autostart = false;
#endif /* #if IS_ENABLED(CONFIG_EEPROM_LOGS) */

LOG_BACKEND_DEFINE(LOG_BACKEND_MILOGGER, logBackendMiLoggerApi, autostart);
```

Рисунок 21

Флаг `autostart` определяется условно в зависимости от опции `Kconfig CONFIG_EEPROM_LOGS`: при отключённой опции бэкенд регистрируется, но не запускается автоматически, что позволяет исключить его из сборки без изменения кода. Функция `backendProcess()` получает управление от диспетчера логирования `Zephyr` для каждого сообщения, прошедшего фильтрацию по уровню. Извлечение текстовой части сообщения осуществляется через `cbprintf()` с пользовательским коллбэком `cbprintf_collect_cb`, записывающим символы непосредственно в целевую строку буфера `m_tempBuffer` со смещением `MILOGGER_TMP_BUFFER_INITIAL_OFFSET`, резервирующим место для заголовка `logHeader`.

Таким образом, `MiLogger` представляет собой полноценную производственную реализацию персистентного логирования для встроенного IoT-устройства, решающую комплекс нетривиальных задач: минимизацию влияния на временные характеристики системы посредством двухуровневой буферизации, предотвращение рекурсии и повреждения данных через RAPI-механизмы блокировки, обеспечение целостности данных через двойную схему CRC, и гибкую маршрутизацию сообщений по уровням серьёзности в отдельные области энергонезависимой памяти.

Компонент `JsonParser`: парсинг конфигурации на основе `jsmn`

В разделе 2.1.1 данной работы была рассмотрена библиотека `jsmn` как пример минималистичного токенизатора JSON без динамической аллокации памяти. В практической части проекта `jsmn` применяется в составе компонента `JsonParser` - многоуровневой системы разбора JSON-конфигурации, загружаемой с сервера. Данный компонент наглядно демонстрирует, каким образом паттерн токенизации без копирования строк,

рассмотренный теоретически, реализуется в производственном встроенном коде.

Иерархия классов парсера

Архитектура компонента построена на абстрактном базовом классе `ABSJsonParser`, определяющем общий интерфейс для всех конкретных парсеров. Класс хранит указатель на массив дескрипторов конфигурационных полей `m_cfgData` типа `JsonData` и указатель на разбираемую JSON-строку `m_jsonStr`. Два чисто виртуальных метода - `set()` и `parse()` - образуют контракт, который обязана реализовать каждая производная реализация:

```
using JsonData = pair<const char *, void *>;

class ABSJsonParser
{
public:
    using size_type = uint32_t;
    using value_type = JsonData;

protected:
    JsonData      *m_cfgData{nullptr};
    uint32_t      m_cfgDataLen{0};

    char          *m_jsonStr;

protected:
    virtual void set(void * const pCfgDataField, jsmntok_t * const pKeyToken, jsmntok_t * const pValToken) = 0;
    virtual bool parse() = 0;

public:
    ABSJsonParser(JsonData *cfgData, char *jsonStr);
};
```

Рисунок 22

Тип `JsonData` представляет собой пару (`const char *`, `void *`) - имя JSON-ключа и указатель на соответствующее поле С-структуры конфигурации. Массив таких пар, завершающийся записью с нулевым ключом, образует декларативную таблицу маппинга JSON-ключей на поля структуры, предоставляемую через `DeviceInfo::getConfigData()`. Данный подход полностью отделяет логику парсинга от конкретной структуры конфигурации: добавление нового конфигурационного поля требует лишь добавления записи в таблицу маппинга без изменения кода парсера. От `ABSJsonParser` наследуются два конкретных класса. `JsonParser` реализует разбор плоских и вложенных полей конфигурации узла и владеет статически выделенным массивом токенов `m_tokens[JSON_MAX_TOKENS]` с максимальным размером 256 токенов. `SensorParser` специализируется на разборе секции `sensors` - массива конфигурационных объектов датчиков с поддержкой вложенных массивов сигналов тревоги. Инициализация токенов и стратегия без динамической памяти. Метод `JsonParser::initTokens()` выполняет токенизацию входной JSON-строки посредством `jsmn`:

```

/*****
 * @brief Initialize jsmn tokens for the given JSON string
 *
 * @return True if JSON token initialization was successful
 *****/
bool JsonParser::initTokens()
{
    jsmn_parser parser;
    jsmn_init(&parser);
    const int32_t res = jsmn_parse(&parser, m_jsonStr, strlen(m_jsonStr), m_tokens, JSON_MAX_TOKENS);
    if (res < 0) {
        LOG_ERR("Failed to parse JSON: %d", res);
        return false;
    }
    m_tokensLen = res;
    return true;
}

```

Рисунок 23

Массив `m_tokens` объявлен как поле класса и размещается статически - либо на стеке при локальном создании объекта, либо в статической памяти при создании как глобального объекта. Ни один байт динамической памяти не выделяется. Максимальный размер конфигурационного JSON ограничен константой `JSON_MAX_TOKENS = 256` токенов, что задаёт верхнюю границу потребления памяти на этапе компиляции. Это полностью соответствует принципам разработки для встроенных систем:

детерминированное потребление памяти, верифицируемое статически.

Маппинг токенов на поля конфигурации

Центральной операцией парсинга является поиск соответствия между токеном JSON-ключа и записью в таблице маппинга. Функция `getJsonDataField()` реализует линейный поиск по таблице с использованием `jsoneq()` - функции сравнения токена со строкой без копирования и без нулевого терминатора:

```

/*****
 * @brief Compares a JSON string token with a given string
 *
 * @param pJsonStr Pointer to the JSON string
 * @param pToken   Pointer to the JSON string token
 * @param pStr     The string to compare with the token
 *
 * @return True if the token matches the given string
 *****/
bool jsoneq(const char *pJsonStr, const jsmntok_t *pToken, const char *pStr)
{
    if (pToken->type != JSMN_STRING) {
        return false;
    }

    if ((int)strlen(pStr) != (pToken->end - pToken->start)) {
        return false;
    }

    if (strncmp(pJsonStr + pToken->start, pStr, pToken->end - pToken->start) != 0) {
        return false;
    }

    return true;
}

```

Рисунок 24

Функция работает исключительно с индексами в оригинальной строке, не модифицируя её. Это позволяет производить сравнение без выделения промежуточных буферов и без установки нулевых терминаторов - что особенно важно при работе с большими JSON-строками конфигурации.

Метод parse и обход дерева токенов

Метод `JsonParser::parse()` обходит плоский массив токенов попарно - ключ и значение - итерируя с шагом 2:


```

for (uint32_t i = 1; i < m_tokensLen - 1; i += 2) {

    jsmntok_t * const pKeyToken = &m_tokens[i];
    jsmntok_t * const pValToken = &m_tokens[i + 1];

    if (pKeyToken->type != JSMN_STRING) {
        LOG_ERR("Keys must be strings: %d", i);
        continue;
    }

    if (pValToken->type & (JSMN_STRING | JSMN_PRIMITIVE)) {

        JsonData * const pCfgDataField = getJsonDataField(m_cfgData, m_jsonStr, pKeyToken);

        if (!pCfgDataField) {
            continue;
        }

        this->set(pCfgDataField->second, pKeyToken, pValToken);
    } else if (pValToken->type & (JSMN_OBJECT | JSMN_ARRAY)) {
        const uint32_t p = i + 1;
        i += 2;
        while (i < m_tokensLen && isChildOf(m_tokens, i, p)) {
            ++i;
        }
        i -= 2;
        continue;
    }

    else {
        LOG_ERR("Invalid token type");
    }
}
}

```

Рисунок 25

При обнаружении вложенного объекта или массива (JSMN_OBJECT | JSMN_ARRAY) парсер пропускает все дочерние токены посредством функции isChildOf(), которая поднимается по цепочке родительских ссылок через поле parent структуры jsmntok_t. Это обеспечивает корректную обработку многоуровневых JSON-документов без рекурсии и без построения дерева в памяти. По завершении обхода плоских полей JsonParser::parse() создаёт экземпляр SensorParser и делегирует ему разбор секции sensors.

Метод set и автоматическое определение типа

Метод JsonParser::set() реализует маппинг JSON-значения на C-поле конфигурации с автоматическим определением типа назначения. Для строковых токенов (JSMN_STRING) значение копируется в поле посредством strncpy с ограничением JSON_STR_VAL_MAX_LEN = 64 символа. Для примитивов реализована логика определения типа по содержимому: "true" / "false" маппируются на bool, числовые значения - через strtodf() с последующей проверкой value == (uint32_t)value для различения целочисленных и вещественных значений:

```

if (!strcmp(pVal, "true", 4)) {
    setWithType(pCfgDataField, true);
} else if (!strcmp(pVal, "false", 5)) {
    setWithType(pCfgDataField, false);
} else {
    float value = strtod(pVal, NULL);
    if (value == (uint32_t)value) {
        setWithType(pCfgDataField, (uint32_t)value);
    } else {
        setWithType(pCfgDataField, value);
    }
}
}

```

Рисунок 26

Запись значения в поле конфигурации выполняется шаблонной функцией `setWithType()`, обеспечивающей типобезопасное присваивание через разыменование приведённого указателя:

```

/*****
 * @brief Set the config data field with automatic type deduction
 *
 * @tparam T          Type of the value to set
 * @param pCfgDataField Void pointer to the corresponding config
 *                    data field
 * @param value       The value to set
 * @param offset      Offset in case if the data is a sensor data
 *****/
template <typename T>
static inline void setWithType(void *pCfgDataField, const T& value, const uint32_t offset = 0) {
    T *p = (static_cast<T*>(pCfgDataField) + offset);
    *p = value;
}

```

Рисунок 27

Заключение

Данная дипломная работа посвящена разработке и реализации программного обеспечения для встроенной системы на базе платформы Nordic Semiconductor nRF9161 с использованием операционной системы реального времени Zephyr RTOS. В ходе выполнения работы были решены все задачи, поставленные во введении, и достигнута сформулированная цель - разработка надёжного, эффективного и сопровождаемого программного обеспечения для IoT-устройства с учётом ограничений аппаратных ресурсов, требований реального времени и условий промышленной эксплуатации.

В первой главе были рассмотрены теоретические основы разработки программного обеспечения для встроенных систем. Дано определение встроенной системы и проведена классификация аппаратных платформ - микроконтроллеры, микропроцессоры, системы на кристалле, FPGA и цифровые сигнальные процессоры - с анализом области применения каждого класса. Детально рассмотрена многоуровневая архитектура встроенных систем, охватывающая аппаратный уровень (процессорное ядро, подсистема памяти, шинная инфраструктура AMBA, периферийные устройства) и программный стек (HAL, слой драйверов, RTOS, прикладной уровень). Проанализирован жизненный цикл разработки встроенного ПО - от анализа требований и архитектурного проектирования через реализацию и многоуровневое тестирование до развёртывания с поддержкой FOTA и долгосрочного сопровождения - с рассмотрением применимых моделей жизненного цикла и их соответствия требованиям функциональной безопасности.

Во второй главе проведён анализ инструментов и технологий разработки встроенного ПО. Рассмотрены языки программирования, применяемые в данной области: ассемблер, C, и C++ - с детальным обоснованием доминирующего положения языка C, обусловленного его близостью к аппаратуре и пред-

сказуемостью генерируемого кода. В подразделе 2.1.1 рассмотрена экосистема минималистичных библиотек для C, воплощённая в паттерне single-header библиотек на примере jsmn - токенизатора JSON без динамической аллокации, далее примененного в практической части работы.

Детально проанализированы операционные системы реального времени: фундаментальные концепции планирования задач, примитивы синхронизации, управление памятью и сравнительный обзор распространённых RTOS, завершающийся углублённым рассмотрением архитектуры Zephyr RTOS - её системы конфигурации Kconfig и Device Tree, модели устройств, сетевого стека и средств трассировки. Рассмотрены аппаратные и программные средства отладки и тестирования: интерфейсы JTAG и SWD, программаторы J-Link, подсистема RTT и инструменты статического анализа.

В третьей главе осуществлена практическая реализация программного обеспечения для IoT-устройства сбора данных с датчиков вибрации и расстояния с передачей в облачную инфраструктуру AWS IoT Core через сотовую сеть LTE. Обоснован выбор аппаратной платформы на базе nRF9161 и программной платформы Zephyr RTOS в составе nRF Connect SDK v3.1.0. Спроектирована и реализована многоуровневая архитектура программного обеспечения, включающая следующие ключевые компоненты.

Компонент ConnectionManager реализует управление жизненным циклом сотового соединения и MQTT-сессии в виде иерархического конечного автомата с состояниями Init, NetworkUp, MqttUp, Connected и Disconnected, обеспечивающего автоматическое восстановление соединения при нестабильном сотовом покрытии.

Компонент SysCtl реализует центральную управляющую логику устройства - инициализацию всех подсистем, основной цикл сбора данных с независи-

мыми временными интервалами для различных типов измерений (стандартные отчёты, RMS, RAW, GNSS), управление обновлением прошивки и интеллектуальное энергосберегающее ожидание на базе ZBus.

Компонент MiLogger реализует пользовательский бэкенд подсистемы логирования Zephyr с персистентным хранением лог-сообщений в энергонезависимой EEPROM-памяти FM24W256. Архитектура компонента включает двухуровневую буферизацию для минимизации влияния на временные характеристики системы, раздельное хранение системных и отладочных логов в выделенных областях памяти с кольцевым буфером для отладочного раздела, двойную схему CRC для обеспечения целостности данных и RAII-механизмы блокировки для предотвращения рекурсии.

Компонент JsonParser реализует многоуровневую систему разбора JSON-конфигурации на основе токенизатора jsmn без динамической аллокации памяти. Иерархия классов с абстрактным базовым парсером ABSJsonParser, конкретными реализациями JsonParser и SensorParser, декларативной таблицей маппинга ключей на поля C-структур и шаблонными функциями типобезопасного присваивания обеспечивает гибкость и расширяемость при строгом соблюдении ресурсных ограничений.

Таким образом, все задачи, поставленные в данной дипломной работе, решены в полном объёме. Разработанное программное обеспечение соответствует требованиям надёжности, производительности и рационального использования аппаратных ресурсов, а применённые архитектурные и технологические решения отражают современный уровень профессиональной разработки встроенного программного обеспечения для промышленных IoT-систем.

Список использованных источников

1. Vardan Grigoryan, Marcelo Guerra - Expert C++
2. Aho – Compilers, Principles, Techniques
3. Grady Booch – Object Oriented Analysis and Design
4. Mohammed Billoo - Embedded Linux Essentials Handbook: Build embedded Linux systems and real-world apps with Yocto, Buildroot, and RPi
5. Scott Meyers – Effective C++
6. Scott Meyers – More Effective C++
7. Scott Meyers – Effective Modern C++
8. Brian Kernighan, Dennis Ritchie - The C Programming Language
9. Emswiler Lippman S., Lajoie J., Moo B. - C++ Primer
10. Alessandro Rubini, Greg Kroah-Hartman - Linux Device Drivers
11. Michael Kerrisk - The Linux Programming Interface
12. Herb Sutter – Exceptional C++
13. Fowler, Martin - UML Distilled: A Brief Guide to the Standard Object Modeling Language
14. Marc Gregoire - Professional C++
15. Erich Gamma – Design Patterns
16. Prem Ponuthorai, Jon Loeliger - Version Control with Git: Powerful Tools and Techniques for Collaborative Software Development
17. Raspberry Pi [source code](#)
18. Rapitasystems: [What really happened on pathfinder aircraft](#)